ĐẶC TẢ HỆ THỐNG · BASELINE V1.0

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Systolic array Output-Stationary 8×8 · DMA theo chuỗi descriptor · Scratchpad ping-pong · VPU

Đề tài: Thiết kế bộ tăng tốc AI tích hợp trong vi xử lý RISC-V

Phiên bản	baseline v1.0 — 11/11/2026
Nguồn	Hợp nhất `spec_1` và `spec_2`; mọi mâu thuẫn đã được giải thành một lựa chọn baseline duy nhất, ghi trong tài liệu quyết định (`decisions/latex/main.tex`)
Nội dung	Kiến trúc, chức năng, mô hình lập trình, luồng vận hành, clock/reset và port top-level của `gemm_accel_top`
Kiến trúc đã chốt	Sáu khối chức năng; DMA theo chuỗi descriptor; scratchpad ping-pong hai bank cho A, B, C, BIAS; systolic array 8×8 PE; `clk` 100 MHz; đầu vào INT8, tích lũy và đầu ra C INT32
Port top-level	**Đã khóa** theo `spec_1`: 44 port (clock, reset, AXI4-Lite slave 17 tín hiệu, AXI4-Full master 25 tín hiệu). Không thêm, bớt hay đổi tên

Mục lục

1 Tổng quan **4**
1.1 Phạm vi chức năng . . . 4
1.2 Quy ước . . . 4
1.3 Kiến trúc tổng thể . . . 5
1.4 Module . . . 5
1.5 Tham số hệ thống . . . 8
1.6 Clock và reset . . . 9
1.7 Tín hiệu giao tiếp . . . 9

2 Đặc tả thanh ghi **13**
2.1 Tổng hợp thanh ghi . . . 13
2.2 Thanh ghi điều khiển (CTRL_REG) . . . 13
2.3 Thanh ghi trạng thái (STATUS_REG) . . . 14
2.4 Cấu hình phần cứng (HW_CONFIG) . . . 14
2.5 Tổng số chu kỳ BUSY của tác vụ (PERF_TOTAL) . . . 15
2.6 Số chu kỳ LOAD của tác vụ, không tính FETCH (PERF_LOAD) . . . 15
2.7 Số chu kỳ COMPUTE của tác vụ (PERF_COMPUTE) . . . 16
2.8 Số chu kỳ STORE_C của tác vụ (PERF_STORE) . . . 16
2.9 Con trỏ descriptor đầu (DESC_PTR) . . . 17
2.10 Địa chỉ descriptor lỗi (ERR_DESC_ADDR) . . . 17
2.11 Số descriptor hoàn tất (DESC_DONE_CNT) . . . 18
2.12 Mã lỗi . . . 18
2.13 Yêu cầu thanh ghi . . . 19
2.14 Ngữ nghĩa tác vụ . . . 19
2.15 Quy tắc cho firmware . . . 20

3 Mô tả chức năng và dữ liệu **20**
3.1 Phép toán . . . 20
3.2 Kiểu dữ liệu, dấu và tràn . . . 21
3.3 Kích thước ma trận và căn chỉnh . . . 21
3.4 Bố trí dữ liệu trong DRAM . . . 22
3.5 Mô hình tham chiếu . . . 23
3.6 Ma trận lớn hơn 32 . . . 23
3.7 Thông lượng . . . 23

4 Luồng vận hành **24**
4.1 Các pha của một descriptor . . . 24
4.2 Luồng một tác vụ . . . 25
4.3 Quyền truy cập scratchpad . . . 25
4.4 Xử lý lỗi . . . 26
4.5 Reset và soft reset . . . 26

5 Chuỗi descriptor và scratchpad ping-pong 27

5.1 Nguyên tắc . . . 27
5.2 DMA theo chuỗi descriptor . . . 27
5.3 Scratchpad ping-pong . . . 29

6 Tích hợp các module 31

6.1 Kết nối giữa các module . . . 31
6.2 Quy tắc tích hợp . . . 32
6.3 Quy tắc giao tiếp . . . 33
6.4 Xử lý lỗi giữa các module . . . 33
6.5 Kiểm chứng ở mức hệ thống . . . 34

1 Tổng quan

`gemm_accel_top` là bộ tăng tốc nhân ma trận INT8 gắn vào SoC Chipyard qua hai cổng AXI: một AXI4-Lite slave cho CPU cấu hình, một AXI4-Full master để tự đọc/ghi DRAM. CPU dựng một *chuỗi descriptor* trong DRAM, ghi `DESC_PTR`, ghi START, rồi chờ bằng polling. Accelerator tự đọc từng descriptor, nạp A và B vào scratchpad, tính $C = A \times B$ trên mảng systolic 8×8 PE, hậu xử lý tùy descriptor, rồi ghi kết quả ra DRAM. Thiết kế gồm đúng sáu khối chức năng, một miền clock `clk` 100 MHz, scratchpad ping-pong hai bank để nạp, tính và ghi của các descriptor kế nhau chạy chồng nhau.

1.1 Phạm vi chức năng

- Mỗi tác vụ là một chuỗi descriptor. Mỗi descriptor tính một phép GEMM: A là M×K, B là K×N, kiểu INT8 có dấu; tích lũy INT32; M $\leq$ 32, N $\leq$ 32, K $\leq$ 32.
- Kết quả ra DRAM ở một trong hai dạng: C (INT32, không hậu xử lý) hoặc Y (INT8, sau bias, requantize, ReLU tùy descriptor, kẹp). Dạng nào do bit `POST_EN` của descriptor quyết định.
- CPU dựng descriptor trong DRAM, ghi `DESC_PTR`, ghi START, rồi chờ kết quả bằng polling. DMA đọc/ghi DRAM qua AXI4-Full, hai kênh độc lập.
- Nạp, tính và ghi của các descriptor kế nhau chạy chồng trên scratchpad ping-pong.
- Dataflow của systolic array là **Output-Stationary**: PE(r, c) giữ bộ cộng dồn của $C[i_0 + r][j_0 + c]$ suốt k = 0 … K−1; A chảy sang đông, B chảy xuống nam, mỗi chu kỳ một PE; tile xong thì xả kết quả theo hàng sang VPU (D-30).
- Ngoài phạm vi (không hỗ trợ): ngắt, $C \mathrel{+}= A \times B$, ma trận lớn hơn 32 trong một descriptor, các dataflow khác (Weight-Stationary, Input-Stationary), bộ nhớ ảo, GEMM lập trình trực tiếp bằng thanh ghi MMIO.

QUYẾT ĐỊNH BASELINE D-01
Hai nguồn khác nhau ở mô hình lập trình: `spec_1` chỉ có chế độ chuỗi descriptor và dự trữ offset 0x08…0x24, còn `spec_2` còn giữ thêm một chế độ *single* lập trình bằng các thanh ghi `MATRIX_DIM`, `BASE_ADDR_IN`, `BASE_ADDR_WEIGHT`, `BASE_ADDR_OUT`, `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT` tại các offset đó, cùng `CTRL[0]` = START, `CTRL[2]` = START_CHAIN và `STATUS[3]` = MODE. Baseline chọn `spec_1`: **chỉ có chế độ chuỗi descriptor**. Lý do: kiến trúc đã chốt là descriptor-based DMA; chế độ register-programmed là một nhánh thiết kế thứ hai làm đúng cùng một việc, nhân đôi đường cấu hình trong Register File và Main FSM (thêm tám bus cấu hình, mỗi bus ứng với một thanh ghi RW, cùng tín hiệu `start_kind` trên giao diện RF→FSM) và buộc STATUS mang thêm một bit chế độ. Tác động: 0x08…0x24 là Reserved; `CTRL[2]` là bit START duy nhất; `STATUS[3]` dự trữ, đọc 0; không có tín hiệu `start_kind`.

1.2 Quy ước

- M, N, K: A là M×K, B là K×N, C và Y là M×N. P: kích thước array vật lý (P×P PE), chốt bằng 8. *word* = 32 bit; *beat* = một lần truyền AXI 32 bit; *tile* = khối P×P của C mà array tính trong một lượt.
- *Tác vụ*: từ khi START được chấp nhận đến khi STATUS báo DONE hoặc ERROR. *Job*: một descriptor trong chuỗi. Tham số của job nằm trong descriptor. `DESC_PTR` được chụp lúc START được chấp nhận.
- *Pha*: một bước của job — FETCH, kiểm tra, LOAD_BIAS, LOAD_A, LOAD_B, COMPUTE, STORE_C.
- Số hex viết dạng 0x6000_0000; số nhị phân dạng 4'b1111. Thời gian quy đổi theo chu kỳ `clk` 10,0 ns.
- Chiều tín hiệu ở biên module nội bộ: `i_*` là input, `o_*` là output. Tín hiệu AXI giữ nguyên tên `s_axi_*` và `m_axi_*`.

1.3 Kiến trúc tổng thể

Dữ liệu đi một chiều: DRAM → DMA → scratchpad → array → VPU → scratchpad → DMA → DRAM. Điều khiển đi vòng: CPU → Register File → Main FSM → DMA và array, rồi về Main FSM bằng các xung done/error. `gemm_accel_top` không chứa logic dữ liệu và không chứa FSM; nó chỉ đấu nối sáu instance và đặt bộ đồng bộ reset hai flip-flop.

Kiến trúc được vẽ thành hai hình, mỗi hình trên một trang nằm ngang riêng. Hình 1.1 là luồng dữ liệu: DMA, scratchpad, array và VPU cùng các giao diện dữ liệu giữa chúng; các cổng điều khiển của bốn khối này chỉ vẽ thành bó dây ngắn ghi nhóm tên port, đi tới `u_main_fsm`. Hình 1.2 là luồng điều khiển: Register File, Main FSM, các lệnh và trạng thái giữa FSM với bốn khối dữ liệu, net `soft_clr` và bộ đồng bộ reset; cổng dữ liệu của bốn khối không vẽ lại ở hình này. Mỗi dây là một bó tín hiệu, ghi nhóm tên port ở hai đầu; danh sách đầy đủ thành viên và port map nằm trong model (`<descid="model">`) của từng file SVG.

Bảng 1.1: Luồng dữ liệu và luồng điều khiển qua sáu khối

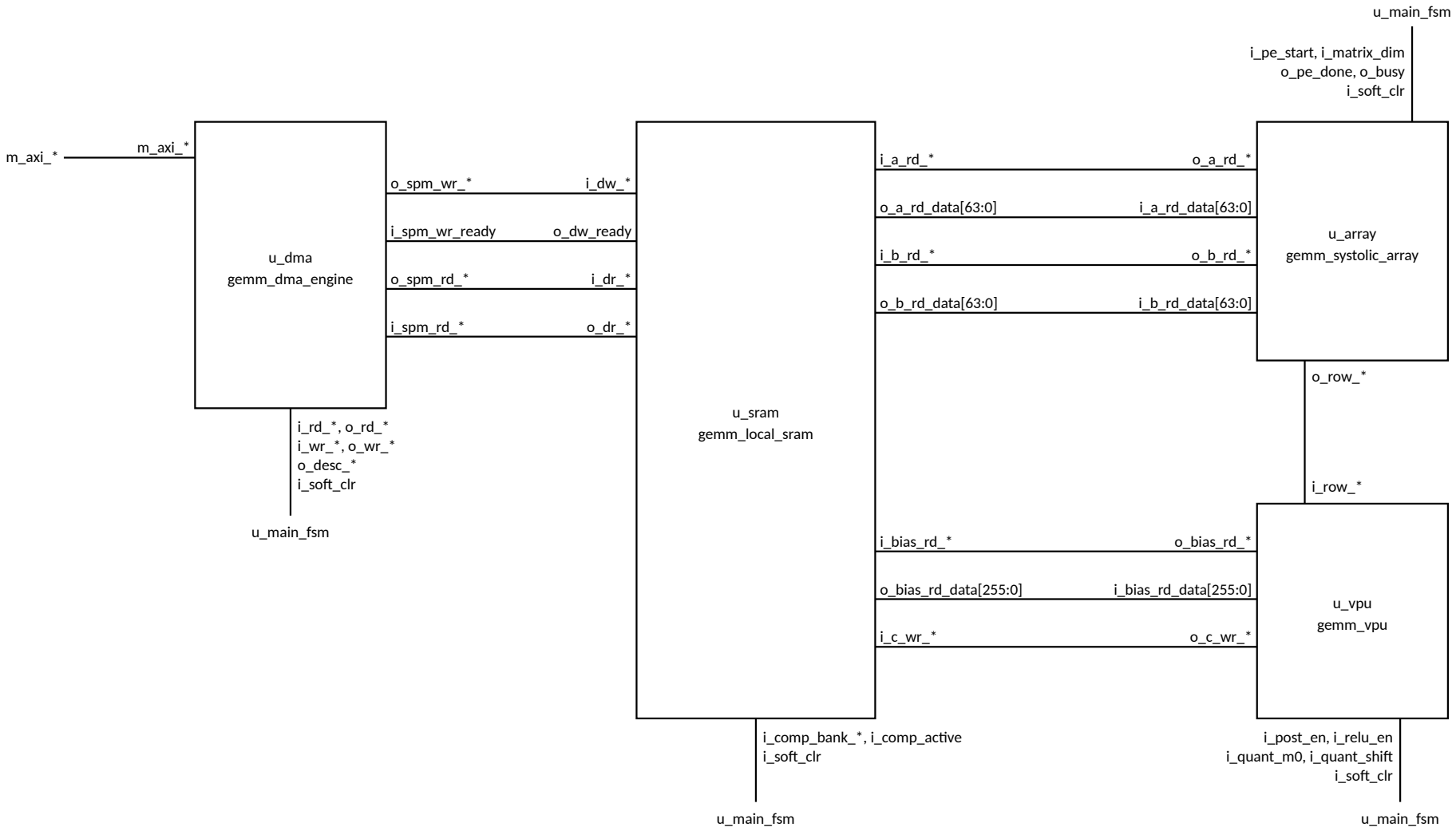
Chặng	Nội dung
CPU → RF	Ghi `DESC_PTR` và `CTRL[2]` qua AXI4-Lite; đọc `STATUS`, `PERF_*`, `ERR_DESC_ADDR`, `DESC_DONE_CNT`
RF → FSM	`o_start_pulse`, `o_soft_req`, `o_cfg_desc_ptr`
FSM → DMA	Lệnh kênh đọc (`o_rd_*`: FETCH, LOAD_BIAS, LOAD_A, LOAD_B) và lệnh kênh ghi (`o_wr_*`: STORE_C)
DRAM → DMA → SRAM	Kênh đọc AXI nhận beat 32 bit, ghi vào bank của buffer A, B hoặc BIAS. Descriptor không vào scratchpad: DMA trả 16 word về FSM qua `o_desc_valid`, `o_desc_idx`, `o_desc_data`
SRAM → Array	Array đọc vector A (`o_a_rd_*`) và B (`o_b_rd_*`), mỗi vector 64 bit, dữ liệu về sau đúng một chu kỳ
Array → VPU	Luồng hàng kết quả: `o_row_valid`, `o_row_data` 256 bit, `o_row_idx`, `o_row_col`, `o_row_mask`. Không có đường ready
VPU → SRAM	VPU đọc bias (`o_bias_rd_*`) và ghi bank C (`o_c_wr_*`, 256 bit, có mask)
SRAM → DMA → DRAM	Kênh ghi đọc bank C qua `o_spm_rd_*`, đẩy qua WFIFO rồi phát beat W ra AXI
FSM → RF	`o_done_pulse`, `o_err_pulse`, `o_err_code`, `o_perf_*`, `o_soft_busy`, `o_err_desc_addr`, `o_desc_done_cnt`
FSM → mọi khối	`o_soft_clr`: xung xóa đồng bộ một chu kỳ tới RF, DMA, SRAM, Array, VPU

1.4 Module

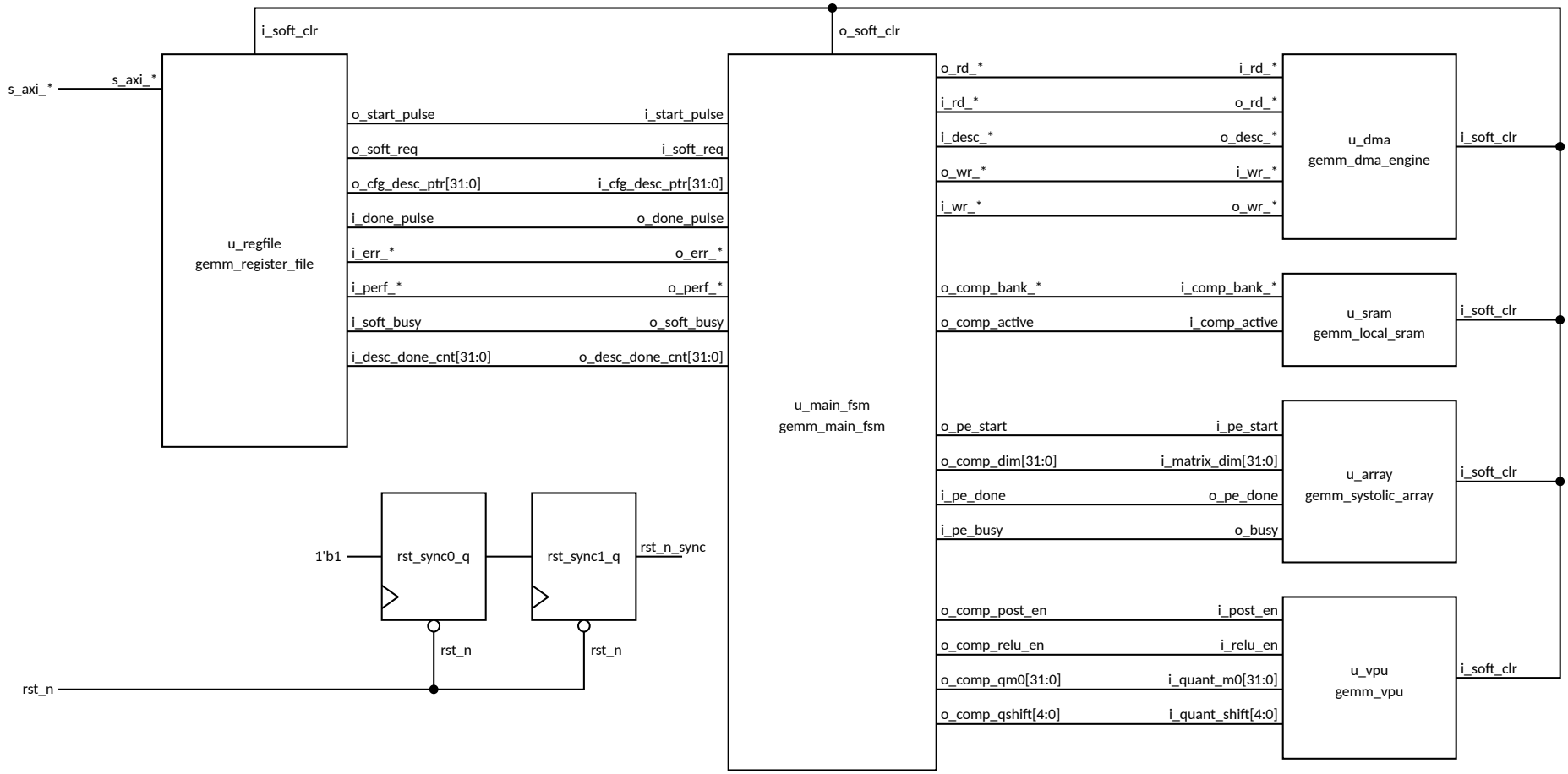
Bảng 1.2: Module trong `gemm_accel_top`

Module	Instance	Vai trò
`gemm_register_file`	`u_regfile`	AXI4-Lite slave; `DESC_PTR`, trạng thái, bộ đếm; chấp nhận START và SOFT_RESET
`gemm_main_fsm`	`u_main_fsm`	Bộ tuần tự duy nhất: kiểm tra descriptor; lập lịch ba engine Đọc, Tính, Ghi; ra lệnh cho DMA và array; báo DONE/ERROR; điều phối soft reset
`gemm_dma_engine`	`u_dma`	AXI4-Full master, hai kênh độc lập; FETCH descriptor, LOAD_A, LOAD_B, LOAD_BIAS, STORE_C
`gemm_local_sram`	`u_sram`	Bốn buffer A, B, C, BIAS; mỗi buffer hai bank ping-pong; mỗi bank chia sub-bank một cổng cho array và VPU
`gemm_systolic_array`	`u_array`	Mảng 8×8 PE (`gemm_pe`) output-stationary, làm lệch dữ liệu, bộ tuần tự tile, xả kết quả theo hàng
`gemm_vpu`	`u_vpu`	Hậu xử lý trên đường xả kết quả: bias, requantize, ReLU, kẹp INT8; đi thẳng khi `POST_EN` = 0

Ngoài sáu module trên, `gemm_accel_top` chỉ có một bộ đồng bộ reset 2 flip-flop. Không có khối thứ bảy, không



Hình 1.1: Luồng dữ liệu của `gemm_accel_top`: DRAM → DMA → scratchpad (A, B, BIAS) → array → VPU → scratchpad (C) → DMA → DRAM



Hình 1.2: Luồng điều khiển của `gemm_accel_top`: CPU → Register File → Main FSM → bốn khối dữ liệu, net `soft_clr` và bộ đồng bộ reset hai flip-flop

có logic dữ liệu ở mức top.

1.5 Tham số hệ thống

Bảng 1.3: Tham số hệ thống

Tham số	Giá trị	Ghi chú
PE_ARRAY_SIZE (P)	8	Giá trị hợp lệ duy nhất; giá trị khác bị chặn lúc elaboration
Số PE	64	P × P
Q	2	Số sub-bank của buffer B; dẫn xuất P/4
LP	3	$\log_2 P$
AB_VEC_WIDTH	64 bit	Vector A và B đọc từ SRAM; dẫn xuất 8·P
ROW_VEC_WIDTH	256 bit	Hàng kết quả array → VPU → bank C; dẫn xuất 32·P
MAX_DIM	32	Giới hạn M, N, K
AXI_DATA_WIDTH	32	Cả AXI4-Lite và AXI4-Full
MAX_BURST_BEATS	16	Baseline chốt 16 (D-36); miền tham số 1 … 256
WFIFO_DEPTH	4	FIFO ghi của DMA, 4 × 32 bit (localparam)
MUL_STAGES	1	Số tầng thanh ghi của bộ nhân requantize trong VPU
VPU_LATENCY (LAT)	4	$3 +$ MUL_STAGES; array dùng cùng giá trị làm L_VPU
NB	2	Số bank mỗi buffer (ping-pong)
Buffer A / B / C / BIAS	256 / 256 / 1024 / 32 word mỗi bank	Hai bank mỗi buffer; 1568 word mỗi bank; tổng 3136 word = 12,25 KiB
SRAM_RD_LAT	1 chu kỳ	Cổng phía array, VPU và DMA
MMIO_BASE	0x6000_0000, vùng 4 KiB	
DRAM_BASE	0x8000_0000	
Burst outstanding	1 đọc + 1 ghi	Mỗi kênh tối đa một burst chưa kết thúc
TARGET_CLK_FREQ	100 MHz	Mục tiêu tổng hợp; chu kỳ 10,0 ns

QUYẾT ĐỊNH BASELINE D-04

spec_1 cho PE_ARRAY_SIZE nhận 4 hoặc 8 và khẳng định kết quả không phụ thuộc P; spec_2 chốt P = 8 cố định. Baseline chốt **P = 8**, giá trị khác bị chặn lúc elaboration. Lý do: kiến trúc đã chốt là PE 8×8, và P = 4 là một nhánh thiết kế thứ hai kéo theo hai bộ độ rộng bus (32 bit và 128 bit cho vector A/B, 128 bit và 256 bit cho hàng kết quả), hai lịch skew và hai công thức timing trong bốn module. Tác động: Q, LP, AB_VEC_WIDTH, ROW_VEC_WIDTH trở thành hằng số; HW_CONFIG[7:0] luôn đọc ra 8.

QUYẾT ĐỊNH BASELINE D-05

spec_1 cho MUL_STAGES nhận 1 hoặc 2 và đặt L_VPU của array mặc định 0 trong khi vẫn yêu cầu L_VPU = LAT — hai câu này mâu thuẫn nhau. spec_2 chốt MUL_STAGES = 1, LAT = 4. Baseline chốt **MUL_STAGES = 1, LAT = L_VPU = 4**, lấy gemm_pkg::VPU_LATENCY làm nguồn duy nhất; array và VPU đều kiểm giá trị này lúc elaboration. Lý do: ở 100 MHz (10 ns) một tầng cho phép nhân 32×32 → 64 bit; MUL_STAGES = 2 chỉ cần khi đẩy tần số cao hơn. Chọn 1 giữ LAT nhỏ nhất, giảm chiều sâu đường ống VPU và khớp mặc định của spec_2. Nếu tổng hợp không đạt 100 MHz, cách sửa đã định trước là đặt MUL_STAGES = 2 (LAT = L_VPU = 5): đây là thay đổi tham số, không phải thay đổi kiến trúc.

1.6 Clock và reset

Bảng 1.4: Clock và reset của hệ thống

Hạng mục	Quy định
Miền clock	Đúng một miền, `clk`. Mọi flip-flop cập nhật tại cạnh lên. Không có CDC, không có clock gating, không có clock chia
Tần số mục tiêu	100 MHz, chu kỳ danh định 10,0 ns, duty 50 %
Đường tổ hợp dài nhất đã biết	Bộ nhân 32×32 → 64 bit của VPU (`MUL_STAGES` = 1) và MAC INT8 một chu kỳ của `gemm_pe`
Reset ngoài	`rst_n`, tích cực mức thấp, kích hoạt bất đồng bộ
Đồng bộ reset	`gemm_accel_top` có bộ đồng bộ 2 flip-flop: bật bất đồng bộ, nhả đồng bộ với `clk`. Đầu ra `rst_n_sync` cấp cho cả sáu module
Độ trễ nhả reset	`rst_n_sync` lên cao sau đúng 2 cạnh lên của `clk` kể từ khi `rst_n` lên cao
Soft reset	`soft_clr`: xung xóa **đồng bộ** một chu kỳ do Main FSM phát. Không đi vào chân reset của bất kỳ flip-flop nào
Nội dung scratchpad	Không bị xóa bởi `rst_n` cũng không bởi `soft_clr`; coi là không xác định

QUYẾT ĐỊNH BASELINE D-20, D-21, D-22
Không nguồn nào nêu chu kỳ clock; kiến trúc chốt đặt 100 MHz, nên baseline lấy chu kỳ danh định 10,0 ns và quy đổi mọi số đo thời gian theo đó (D-20). Nguồn nêu bộ đồng bộ 2 flip-flop nhưng không nêu giá trị khởi tạo: baseline chọn **cả hai flip-flop có reset bất đồng bộ về 0**, nên `rst_n_sync` xuống thấp ngay khi `rst_n` xuống, không cần clock chạy — bảo đảm accelerator vào trạng thái reset khi SoC chưa có clock ổn định (D-21). Quy tắc hiện thực kèm theo: flip-flop trạng thái dùng `always_ff@(posedgeclkornegedgerst_n_sync)`; flip-flop chỉ chứa dữ liệu (`acc` và `res` của PE, nội dung RAM, thanh ghi dữ liệu trong đường ống VPU) dùng `always_ff@(posedgeclk)` và không có giá trị reset. Nội dung scratchpad sau reset là **không xác định**, không có mạch xóa (D-22); firmware không được đọc vùng đích trước DONE.

1.7 Tín hiệu giao tiếp

Tín hiệu giao tiếp của `gemm_accel_top` **đã khóa**: clock, reset, một cổng AXI4-Lite slave (17 tín hiệu) và một cổng AXI4-Full master (25 tín hiệu), tổng 44 port. Baseline giữ nguyên hai bảng tín hiệu AXI của `spec_1`: không thêm, bớt, đổi tên, đổi độ rộng hay đổi chiều bất kỳ port nào. `spec_2` mô tả đúng cùng 44 port đó (REQ-IF-001, IF-001 và IF-002), nên ở điểm này hai nguồn không mâu thuẫn. Mọi thay đổi về sau chỉ được xảy ra ở port nội bộ giữa các module.

1.7.1 Tham số

Bảng 1.5: Tham số của `gemm_accel_top`

Tham số	Giá trị	Ý nghĩa
`PE_ARRAY_SIZE`	8	P; giá trị hợp lệ duy nhất là 8 (quyết định baseline D-04). Giá trị khác bị chặn lúc elaboration
`MAX_BURST_BEATS`	16	Số beat tối đa mỗi burst, miền 1 … 256; baseline chốt 16 (D-36)

Danh sách port cố định cho mọi giá trị tham số hợp lệ. Giá trị tham số ngoài miền cho phép phải bị chặn lúc elaboration.

1.7.2 Danh sách tín hiệu

Bảng 1.6: Port clock, reset và AXI4-Lite slave (MMIO)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset ngoài tích cực mức thấp, kích hoạt bất đồng bộ. Top dùng hai flip-flop đồng bộ cạnh nhả trước khi phân phối reset tới các module.
`s_axi_awaddr`	32	Input	Địa chỉ byte của thanh ghi cần ghi. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
`s_axi_awvalid`	1	Input	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
`s_axi_awready`	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
`s_axi_wdata`	32	Input	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
`s_axi_wstrb`	4	Input	Cho phép ghi từng byte của thanh ghi RW. Bit b = 1 cho phép ghi byte b; bit b = 0 giữ byte đó. Các bit lệnh CTRL chỉ có hiệu lực khi bit 0 bằng 1.
`s_axi_wvalid`	1	Input	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
`s_axi_wready`	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
`s_axi_bresp`	2	Output	Mã phản hồi ghi. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng BVALID.
`s_axi_bvalid`	1	Output	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
`s_axi_bready`	1	Input	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1.
`s_axi_araddr`	32	Input	Địa chỉ byte của thanh ghi cần đọc. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
`s_axi_arvalid`	1	Input	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
`s_axi_arready`	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
`s_axi_rdata`	32	Output	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
`s_axi_rresp`	2	Output	Mã phản hồi đọc. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng RVALID.

Bảng 1.6: Port clock, reset và AXI4-Lite slave (MMIO) (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
s_axi_rvalid	1	Output	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
s_axi_rready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.

Bảng 1.7: Port AXI4-Full master (DMA)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
m_axi_araddr	32	Output	Địa chỉ byte đầu tiên của burst đọc. Địa chỉ căn 4 byte. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
m_axi_arlen	8	Output	Số beat của burst đọc trừ 1. Hợp lệ cùng ARVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
m_axi_arsize	3	Output	Kích thước mỗi beat của burst đọc. Luôn bằng 3'b010, tương ứng 4 byte.
m_axi_arburst	2	Output	Kiểu burst đọc. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
m_axi_arvalid	1	Output	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
m_axi_arready	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
m_axi_rdata	32	Input	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
m_axi_rresp	2	Input	Mã phản hồi beat đọc. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng RVALID.
m_axi_rlast	1	Input	Đánh dấu beat cuối burst đọc khi RVALID = 1. DMA so sánh vị trí với ARLEN + 1; đến sớm hoặc muộn đều là lỗi.
m_axi_rvalid	1	Input	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
m_axi_rready	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.
m_axi_awaddr	32	Output	Địa chỉ byte đầu tiên của burst ghi. Địa chỉ căn 4 byte. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
m_axi_awlen	8	Output	Số beat của burst ghi trừ 1. Hợp lệ cùng AWVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
m_axi_awsize	3	Output	Kích thước mỗi beat của burst ghi. Luôn bằng 3'b010, tương ứng 4 byte.
m_axi_awburst	2	Output	Kiểu burst ghi. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.

Bảng 1.7: Port AXI4-Full master (DMA) (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
m_axi_awvalid	1	Output	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
m_axi_awready	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
m_axi_wdata	32	Output	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
m_axi_wstrb	4	Output	Mặt nạ byte ghi. Luôn bằng 4'b1111 vì DMA ghi nguyên word 32 bit.
m_axi_wlast	1	Output	Đánh dấu beat cuối burst ghi khi WVALID = 1. Bằng 1 ở đúng beat thứ AWLEN + 1 và giữ nguyên trong lúc chờ WREADY.
m_axi_wvalid	1	Output	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
m_axi_wready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
m_axi_bresp	2	Input	Mã phản hồi burst ghi. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng BVALID.
m_axi_bvalid	1	Input	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
m_axi_bready	1	Output	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1. DMA luôn giữ BREADY = 1.

Port top-level

- Tín hiệu giao tiếp của `gemm_accel_top` đúng tên, độ rộng và chiều như hai bảng trên; không có port nào khác, kể cả ngắt
- Danh sách port này cố định cho mọi giá trị tham số hợp lệ
- Giá trị tham số ngoài miền cho phép phải bị chặn lúc elaboration
- Kênh đọc AXI (AR, R) và kênh ghi (AW, W, B) của cổng master do DMA điều khiển; cổng AXI4-Lite slave do Register File điều khiển. Không có đường tổ hợp từ input tới output trên cùng một cổng AXI
- `gemm_accel_top` là Chisel BlackBox, nối hệ thống qua cầu TLToAXI4 (MMIO) và AXI4ToTL (DMA). Tín hiệu AXI phụ (ID, LOCK, CACHE, PROT, QOS) do `GemmWrapper.scala` cấp và không phải port của `gemm_accel_top`
- Top chỉ đấu nối: `s_axi_*` đi thẳng tới `u_regfile`, `m_axi_*` đi thẳng từ `u_dma`, không có logic chèn giữa, độ trễ xuyên qua top là 0 chu kỳ

2 Đặc tả thanh ghi

Firmware dựng chuỗi descriptor trong DRAM, ghi `DESC_PTR`, thực hiện `fence rw,rw`, ghi START (`CTRL[2]`), rồi đọc `STATUS` cho tới khi DONE hoặc ERROR bằng 1. Tham số của từng phép GEMM nằm trong descriptor (mục 5.2), không nằm trong thanh ghi MMIO. Offset 0x08 … 0x24 dự trữ.

2.1 Tổng hợp thanh ghi

Địa chỉ tuyệt đối = `MMIO_BASE` + offset; vùng MMIO 4 KiB tại 0x6000_0000, tức 0x6000_0000 … 0x6000_0FFF. Thanh ghi căn word; phần cứng chỉ giải mã bit địa chỉ [11:2]. RW: đọc/ghi theo byte; RO: chỉ đọc, ghi bị bỏ qua; W: ghi lệnh, đọc luôn bằng 0. Không có bit RW1C. Các offset không liệt kê đọc 0, ghi bị bỏ qua, phản hồi luôn OKAY.

Bảng 2.1: Tổng hợp thanh ghi

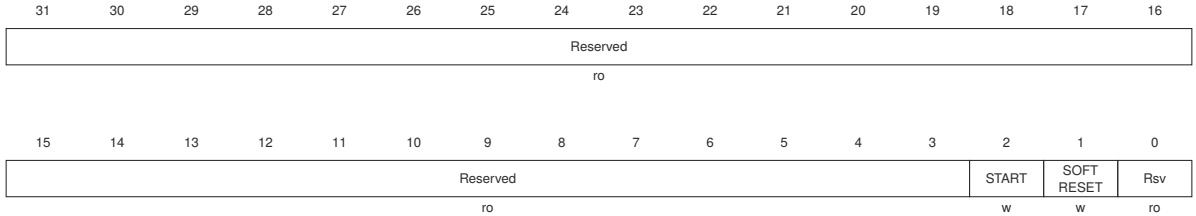
Offset	Tên viết tắt	Tên thanh ghi
0x00	CTRL_REG	Thanh ghi điều khiển
0x04	STATUS_REG	Thanh ghi trạng thái
0x08 … 0x24	—	Reserved
0x28	HW_CONFIG	Cấu hình phần cứng
0x2C	PERF_TOTAL	Tổng số chu kỳ BUSY của tác vụ
0x30	PERF_LOAD	Số chu kỳ LOAD, không tính FETCH
0x34	PERF_COMPUTE	Số chu kỳ COMPUTE
0x38	PERF_STORE	Số chu kỳ STORE_C
0x3C	—	Reserved
0x40	DESC_PTR	Con trỏ descriptor đầu
0x44	ERR_DESC_ADDR	Địa chỉ descriptor lỗi
0x48	DESC_DONE_CNT	Số descriptor hoàn tất
Các địa chỉ khác	—	Reserved

QUYẾT ĐỊNH BASELINE D-01
Offset 0x08 … 0x24 là **Reserved**. `spec_2` đặt tại đây tám thanh ghi RW của chế độ single (`MATRIX_DIM`, `BASE_ADDR_IN`, `BASE_ADDR_WEIGHT`, `BASE_ADDR_OUT`, `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT`); baseline bỏ chế độ đó nên bỏ luôn tám thanh ghi. Mọi tham số của job lấy từ descriptor. Hệ quả cho firmware: ghi vào 0x08 … 0x24 bị bỏ qua và đọc ra 0, phản hồi vẫn OKAY.

2.2 Thanh ghi điều khiển (CTRL_REG)

Địa chỉ offset: 0x00

Giá trị reset: 0x0000_0000



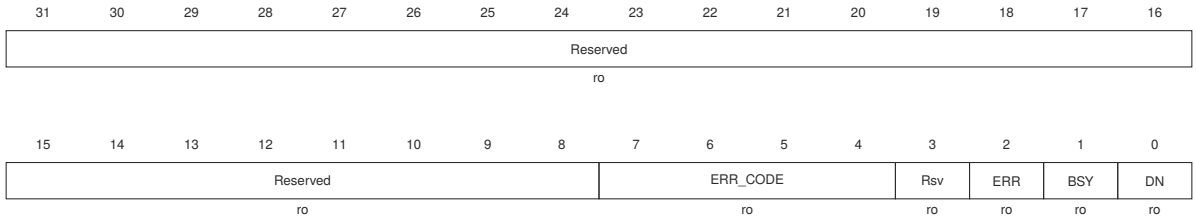
Bảng 2.2: Thanh ghi điều khiển

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:3	Reserved	RO	29'h0	Đọc 0; ghi bị bỏ qua.
2	START	W	1'h0	Ghi 1 bắt đầu chuỗi descriptor tại DESC_PTR. Bit tự xóa, đọc luôn ra 0. Chỉ được chấp nhận khi BUSY = 0; khi bận bị bỏ qua, không báo lỗi.
1	SOFT_RESET	W	1'h0	Ghi 1 yêu cầu soft reset (mục 4.5); bit tự xóa. Ưu tiên cao hơn START trong cùng một lệnh ghi.
0	Reserved	RO	1'h0	Đọc 0; ghi bị bỏ qua.

2.3 Thanh ghi trạng thái (STATUS_REG)

Địa chỉ offset: 0x04

Giá trị reset: 0x0000_0000



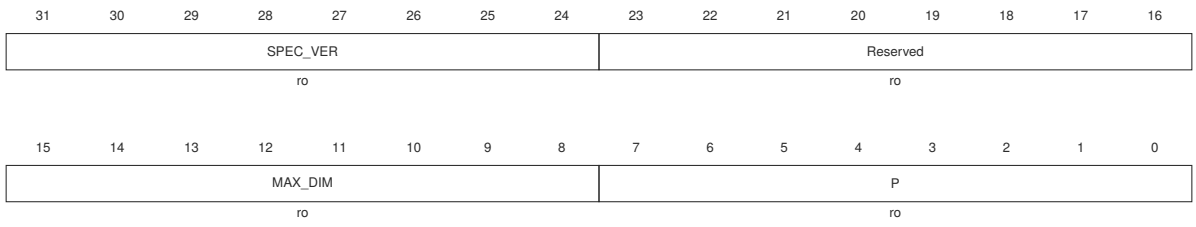
Bảng 2.3: Thanh ghi trạng thái

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:8	Reserved	RO	24'h0	Đọc 0; ghi bị bỏ qua.
7:4	ERR_CODE	RO	4'h0	Mã lỗi của tác vụ; bằng 0 khi ERROR = 0. Bảng mã lỗi ở mục 2.12.
3	Reserved	RO	1'h0	Đọc 0; ghi bị bỏ qua.
2	ERROR	RO	1'h0	Cờ lỗi sticky; xóa khi nhận tác vụ mới, khi soft reset hoặc khi reset.
1	BUSY	RO	1'h0	Tác vụ hoặc chuỗi soft reset đang chạy. Về 0 cùng cạnh clock đặt DONE hoặc ERROR.
0	DONE	RO	1'h0	Sticky; chỉ đặt khi toàn bộ kết quả của chuỗi đã ghi DRAM thành công. Xóa khi nhận tác vụ mới, khi soft reset hoặc khi reset.

2.4 Cấu hình phần cứng (HW_CONFIG)

Địa chỉ offset: 0x28

Giá trị reset: 0x1100_2008



Bảng 2.4: Cấu hình phần cứng

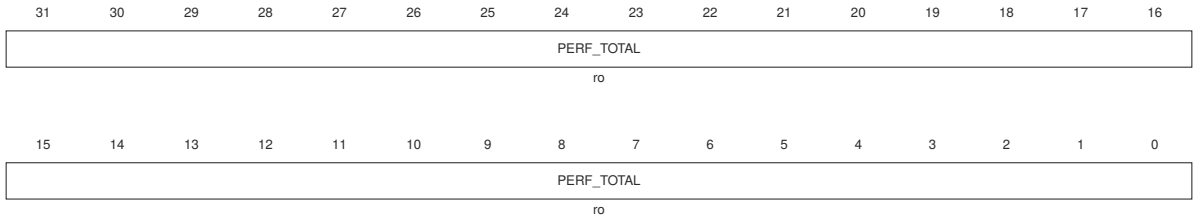
Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:24	SPEC_VER	RO	8'h11	Phiên bản giao diện phần cứng. 0x11: chỉ chuỗi descriptor và scratchpad ping-pong.
23:16	Reserved	RO	8'h0	Đọc 0; ghi bị bỏ qua.
15:8	MAX_DIM	RO	8'd32	Kích thước ma trận tối đa của một descriptor.
7:0	P	RO	8'd8	Kích thước array; luôn bằng 8.

QUYẾT ĐỊNH BASELINE D-03
Hai nguồn cho hai giá trị reset khác nhau: `spec_1` dùng {8'h11, 8'h00, 8'd32, P} với SPEC_VER = 0x11 nghĩa là "chỉ chuỗi descriptor và ping-pong"; `spec_2` dùng 0x1003_2008 với SPEC_VER = 0x10 và thêm hai bit tính năng `HAS_CHAIN` = bit 16, `HAS_PINGPONG` = bit 17. Baseline chọn `spec_1` với P chốt bằng 8, tức giá trị reset **0x1100_2008**. Lý do: khi chuỗi descriptor và ping-pong là chế độ duy nhất (D-01), hai bit `HAS_*` là hằng số 1 và không mang thông tin, trong khi SPEC_VER = 0x11 đã mã hóa đúng khả năng đó bằng một trường duy nhất. Giữ cả hai cách mô tả cùng một việc tạo ra hai nguồn sự thật cho firmware. Tác động: bit [23:16] là Reserved đọc 0; firmware nhận biết khả năng của phần cứng qua SPEC_VER.

2.5 Tổng số chu kỳ BUSY của tác vụ (PERF_TOTAL)

Địa chỉ offset: 0x2C

Giá trị reset: 0x0000_0000



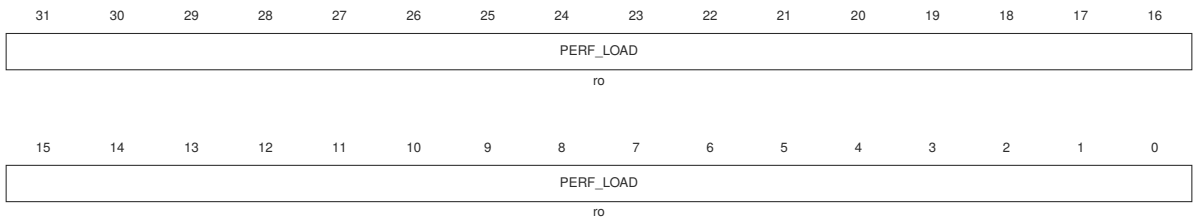
Bảng 2.5: Tổng số chu kỳ BUSY của tác vụ

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_TOTAL	RO	32'h0	Tổng số chu kỳ BUSY của tác vụ. Về 0 ở chu kỳ START được chấp nhận; đếm trong lúc BUSY = 1; bão hòa tại 0xFFFF_FFFF; giữ giá trị sau khi tác vụ kết thúc.

2.6 Số chu kỳ LOAD của tác vụ, không tính FETCH (PERF_LOAD)

Địa chỉ offset: 0x30

Giá trị reset: 0x0000_0000



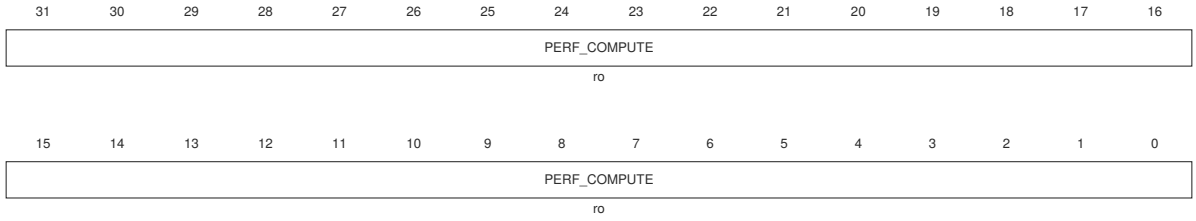
Bảng 2.6: Số chu kỳ LOAD của tác vụ, không tính FETCH

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_LOAD	RO	32'h0	Số chu kỳ LOAD của tác vụ, không tính FETCH. Về 0 ở chu kỳ START được chấp nhận; tăng ở mỗi chu kỳ BUSY = 1 có pha LOAD đang chạy (`i_perf_load` = 1); bão hòa tại 0xFFFF_FFFF; giữ giá trị sau khi tác vụ kết thúc.

2.7 Số chu kỳ COMPUTE của tác vụ (PERF_COMPUTE)

Địa chỉ offset: 0x34

Giá trị reset: 0x0000_0000



Bảng 2.7: Số chu kỳ COMPUTE của tác vụ

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_COMPUTE	RO	32'h0	Số chu kỳ COMPUTE của tác vụ. Về 0 ở chu kỳ START được chấp nhận; tăng ở mỗi chu kỳ BUSY = 1 có pha COMPUTE đang chạy (`i_perf_comp` = 1); bão hòa tại 0xFFFF_FFFF; giữ giá trị sau khi tác vụ kết thúc.

2.8 Số chu kỳ STORE_C của tác vụ (PERF_STORE)

Địa chỉ offset: 0x38

Giá trị reset: 0x0000_0000



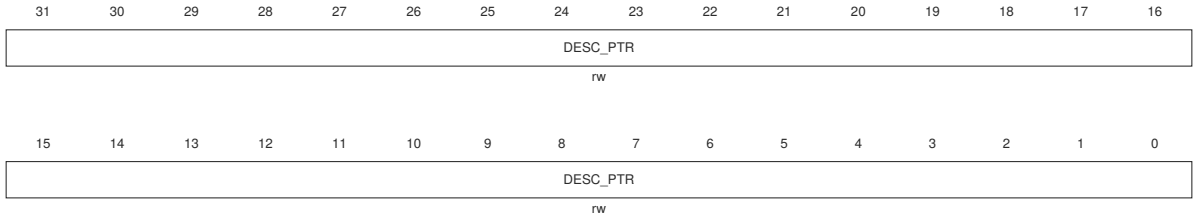
Bảng 2.8: Số chu kỳ STORE_C của tác vụ

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_STORE	RO	32'h0	Số chu kỳ STORE_C của tác vụ. Về 0 ở chu kỳ START được chấp nhận; tăng ở mỗi chu kỳ BUSY = 1 có pha STORE_C đang chạy (`i_perf_store` = 1); bão hòa tại 0xFFFF_FFFF; giữ giá trị sau khi tác vụ kết thúc.

2.9 Con trỏ descriptor đầu (DESC_PTR)

Địa chỉ offset: 0x40

Giá trị reset: 0x0000_0000



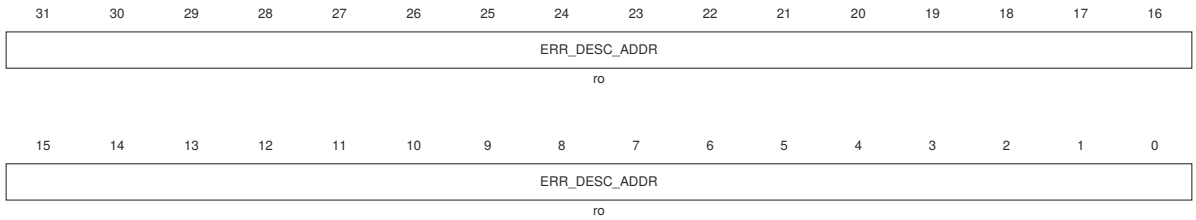
Bảng 2.9: Con trỏ descriptor đầu

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	DESC_PTR	RW	32'h0	Địa chỉ byte của descriptor đầu chuỗi; phải căn 64 byte. Lưu đủ 32 bit. Ghi được bất kỳ lúc nào, kể cả khi BUSY = 1; đọc lại trả về giá trị CPU đã ghi, không phải bản chụp. Giá trị mới áp dụng từ lần START được chấp nhận tiếp theo. Soft reset giữ nguyên thanh ghi này.

2.10 Địa chỉ descriptor lỗi (ERR_DESC_ADDR)

Địa chỉ offset: 0x44

Giá trị reset: 0x0000_0000



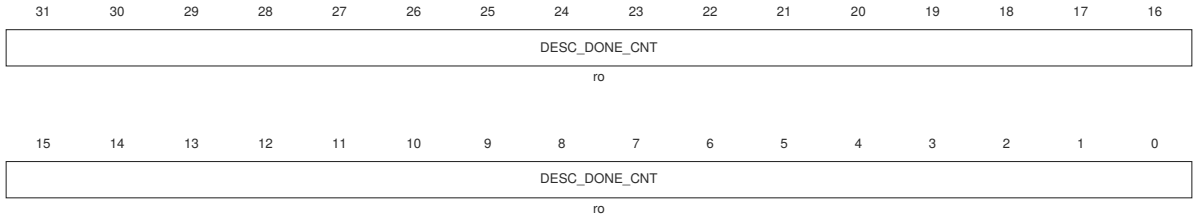
Bảng 2.10: Địa chỉ descriptor lỗi

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	ERR_DESC_ADDR	RO	32'h0	Địa chỉ của descriptor có lỗi sớm nhất theo thứ tự chuỗi. Chỉ có nghĩa khi ERROR = 1. Xóa khi nhận tác vụ mới, khi soft reset hoặc khi reset.

2.11 Số descriptor hoàn tất (DESC_DONE_CNT)

Địa chỉ offset: 0x48

Giá trị reset: 0x0000_0000



Bảng 2.11: Số descriptor hoàn tất

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	DESC_DONE_CNT	RO	32'h0	Tăng 1 mỗi khi STORE_C của một descriptor nhận phản hồi B cuối là OKAY; các descriptor hoàn tất đúng theo thứ tự chuỗi. Khi ERROR = 1, bằng số thứ tự của descriptor lỗi sớm nhất. Xóa khi nhận tác vụ mới, khi soft reset hoặc khi reset.

2.12 Mã lỗi

Bảng 2.12: Mã lỗi `ERR_CODE` (STATUS[7:4])

Giá trị	Ý nghĩa	Khi nào xuất hiện
0	Không lỗi	ERROR = 0
1	Cấu hình sai	Descriptor sai điều kiện hợp lệ ở mục 5.2, hoặc `DESC_PTR` không căn 64 byte
2	Lỗi AXI khi đọc	LOAD_A, LOAD_B hoặc LOAD_BIAS gặp `m_axi_rresp` ≠ OKAY hoặc `m_axi_rlast` sai vị trí
3	Lỗi AXI khi ghi	STORE_C gặp `m_axi_bresp` ≠ OKAY
4	Lỗi AXI khi đọc descriptor	FETCH gặp `m_axi_rresp` ≠ OKAY hoặc `m_axi_rlast` sai vị trí
5 … 15	Dự trữ	Không bao giờ xuất hiện

QUYẾT ĐỊNH BASELINE D-02

`spec_2` định nghĩa thêm mã 5 "tính năng không có", đồng thời tự ghi rằng mã này không bao giờ phát sinh vì chế độ chuỗi luôn bật. Baseline theo `spec_1`: miền mã lỗi là 0 … 4, còn 5 … 15 dự trữ. Lý do: đây là hệ quả trực tiếp của D-01 — khi chỉ có một chế độ thì không tồn tại tình huống "tính năng không có", nên một mã lỗi không thể phát sinh chỉ làm phức tạp bảng mã và phần mềm xử lý lỗi.

2.13 Yêu cầu thanh ghi

Thanh ghi

- Offset, tên, quyền truy cập và giá trị reset đúng theo register map ở trên
- CTRL[2] START: ghi 1 bắt đầu chuỗi descriptor tại DESC_PTR. Bit tự xóa, đọc luôn ra 0. Ghi 0 không có tác dụng. Ghi khi bận bị bỏ qua, không báo lỗi
- CTRL[1] SOFT_RESET: ghi 1 là yêu cầu soft reset (mục 4.5); tự xóa. Một lần ghi CTRL có SOFT_RESET = 1 thì chỉ SOFT_RESET có hiệu lực, START trong cùng lệnh ghi bị bỏ qua
- CTRL[0] dự trữ: đọc 0, ghi bị bỏ qua. Các bit lệnh của CTRL chỉ có hiệu lực khi s_axi_wstrb[0] = 1
- ERR_CODE = 0 khi ERROR = 0. STATUS[3] luôn đọc ra 0
- DESC_PTR ghi được bất kỳ lúc nào, kể cả khi BUSY = 1. Đọc lại trả giá trị CPU đã ghi, không phải bản chụp. Giá trị mới áp dụng từ lần START được chấp nhận tiếp theo
- Thanh ghi RW lưu đủ 32 bit, kể cả bit dự trữ. Bit dự trữ của CTRL và STATUS: ghi bị bỏ qua, đọc ra 0. Offset dự trữ, kể cả 0x08 … 0x24 và 0x3C: đọc ra 0, ghi bị bỏ qua. Ghi vào thanh ghi chỉ đọc bị bỏ qua. Mọi truy cập đều trả s_axi_bresp / s_axi_rresp = 2'b00 (OKAY)
- rst_n đưa mọi thanh ghi về giá trị reset. Soft reset giữ nguyên DESC_PTR và các thanh ghi RW; xóa STATUS, PERF_*, ERR_DESC_ADDR và DESC_DONE_CNT
- PERF_* về 0 ở chu kỳ START được chấp nhận; PERF_TOTAL đếm mọi chu kỳ BUSY = 1, PERF_LOAD, PERF_COMPUTE, PERF_STORE chỉ đếm chu kỳ BUSY = 1 có pha tương ứng đang chạy; dừng khi tác vụ kết thúc, bão hòa ở 0xFFFF_FFFF. Khi các pha chồng nhau, PERF_LOAD + PERF_COMPUTE + PERF_STORE có thể lớn hơn PERF_TOTAL; phần chênh là thời gian chạy chồng
- DESC_PTR được chụp ở chu kỳ START được chấp nhận và giữ ổn định tới hết tác vụ. Nếu bit [5:0] của bản chụp khác 0, tác vụ kết thúc với ERR_CODE = 1 và **không** truy cập bus, **không** truy cập scratchpad

2.14 Ngữ nghĩa tác vụ

Tác vụ

- START chỉ được chấp nhận nếu BUSY = 0 tại chu kỳ Register File thực hiện lệnh ghi CTRL. START khi BUSY = 1 bị bỏ qua, không đổi trạng thái, không báo lỗi
- Ngay chu kỳ chấp nhận: Register File chụp DESC_PTR; xóa DONE, ERROR, ERR_CODE, PERF_*. BUSY = 1 từ chu kỳ kế tiếp. ERR_DESC_ADDR và DESC_DONE_CNT nằm trong Main FSM và được xóa ở cạnh clock cuối chu kỳ có o_start_pulse, tức chậm hơn một chu kỳ; CPU không quan sát được độ lệch này vì mọi lần đọc sau phản hồi B của lệnh ghi CTRL đều tới muộn hơn
- Register File phát o_start_pulse và chụp DESC_PTR trước khi trả phản hồi B cho lệnh ghi CTRL; mọi lần đọc STATUS sau khi lệnh ghi hoàn tất đều thấy tác vụ mới
- DONE = 1 chỉ khi STORE_C của descriptor có LAST = 1 đã nhận phản hồi B cuối là OKAY và không có lỗi nào trong cả chuỗi
- ERROR được đặt khi đã có lỗi và mọi engine đã yên. Gọi f là descriptor lỗi sớm nhất theo thứ tự chuỗi: không pha mới nào của descriptor có chỉ số $\geq$ f được bắt đầu; mọi descriptor đứng trước f chạy xong, kể cả STORE_C. ERR_DESC_ADDR là địa chỉ của f; DESC_DONE_CNT bằng f; vùng đích trong DRAM của descriptor $\geq$ f không xác định
- BUSY về 0 ở đúng cạnh clock mà DONE hoặc ERROR được đặt. Sau khi chấp nhận và cho tới tác vụ kế tiếp hoặc soft reset, mọi lần đọc STATUS thấy đúng một trong ba bit BUSY, DONE, ERROR bằng 1
- o_done_pulse và o_err_pulse không bao giờ xuất hiện trong cùng một chu kỳ
- DONE và ERROR giữ giá trị (sticky); chỉ bị xóa khi tác vụ mới được chấp nhận, khi soft reset hoặc khi rst_n
- Sau DONE hoặc ERROR, firmware được phép ghi START ngay, không cần soft reset
- Tín hiệu "xong" nội bộ (o_rd_done, o_wr_done của một lệnh DMA, o_pe_done của array) không hiện ra cho CPU; chỉ STATUS.DONE báo toàn bộ tác vụ xong

2.15 Quy tắc cho firmware

Firmware

- Trước lệnh ghi START, firmware ghi descriptor và dữ liệu vào DRAM, ghi `DESC_PTR`, rồi thực hiện `fencerw,rw`
- Firmware chờ bằng polling cho tới khi DONE hoặc ERROR bằng 1, rồi thực hiện `fencerw,rw` trước khi đọc kết quả
- Đường DMA được giả định nhất quán cache với CPU. Nếu hệ thống tích hợp không bảo đảm điều đó, firmware phải flush dòng cache của descriptor, A, B và bias trước START, và hủy dòng cache của vùng đích sau DONE
- Firmware không sửa descriptor và không ghi vào vùng DRAM của chuỗi đang chạy

```
#define GEMM_BASE  0x60000000u
#define REG(off)   (*(volatile uint32_t *)(GEMM_BASE + (off)))
#define ST_DONE    (1u << 0)
#define ST_ERROR   (1u << 2)

int gemm_run(uint32_t desc) {
    REG(0x40) = desc;                    /* DESC_PTR, căn 64 byte */
    asm volatile ("fence rw,rw");
    REG(0x00) = 4u;                      /* CTRL[2] = START */
    uint32_t s;
    do { s = REG(0x04); } while (!(s & (ST_DONE | ST_ERROR)));
    asm volatile ("fence rw,rw");
    return (s & ST_ERROR) ? (int)((s >> 4) & 0xFu) : 0;
}
```

3 Mô tả chức năng và dữ liệu

Mỗi descriptor tính $C = A \times B$ trên INT8, tích lũy INT32. Khi `POST_EN` = 0, C (INT32) được ghi ra DRAM. Khi `POST_EN` = 1, VPU biến C thành $Y = \text{kẹp}(\text{ReLU}(\text{requantize}(C + bias)))$ ở INT8 và Y được ghi ra DRAM. Mọi phép tính là số nguyên và kết quả phải khớp từng bit với mô hình tham chiếu ở mục 3.5.

3.1 Phép toán

Phép toán

- $C[i][j] = \sum_{k=0}^{K-1} A[i][k] \cdot B[k][j]$, với 0 ≤ i < M, 0 ≤ j < N
- A, B là INT8 bù 2. C và bias là INT32 bù 2. Y là INT8 bù 2. Tích INT8 × INT8 tính đủ 16 bit có dấu rồi mở rộng dấu lên 32 bit; cộng dồn trên INT32, wrap mod 2^{32} nếu tràn
- Khi `POST_EN` = 1, với mỗi phần tử:

  $acc = C[i][j] + bias[j]$ (INT32, wrap mod 2^{32})
  $prod = acc \cdot M_0$ (tích có dấu 64 bit)
  $y = (prod + 2^{30+sh}) \gg (31 + sh)$ (dịch số học, 64 bit)
  nếu `RELU_EN` = 1: $y = \max(y, 0)$
  $Y[i][j] = \min(\max(y, -128), 127)$
- M_0 (`QM0` của descriptor) là INT32 có dấu, mọi giá trị đều hợp lệ. sh (`QSHIFT`) là số không dấu 0 … 31. Một cặp (M_0, sh) dùng cho một descriptor. bias là vector N phần tử INT32, cộng theo cột
- Kết quả ghi đè vùng đích trong DRAM; accelerator không đọc giá trị cũ của vùng đích
- Chỉ hỗ trợ nhân ma trận và hậu xử lý đã mô tả; không có $C \mathrel{+}= A \times B$, softmax hoặc pooling

HỆ QUẢ
$|a \cdot b| \le 2^{14}$ và K $\le$ 32 nên $|C| \le 2^{19}$: C không bao giờ tràn. $|prod| \le 2^{62}$ và $2^{30+sh} \le 2^{61}$ nên phép tính hậu xử lý 64 bit không bao giờ tràn. Làm tròn là nửa lên (về phía $+\infty$). Phần mềm nên chọn M_0 trong $[2^{30}, 2^{31})$ để giữ độ chính xác.

3.2 Kiểu dữ liệu, dấu và tràn

Bảng 3.1: Kiểu dữ liệu và quy tắc tràn

Đại lượng	Kiểu	Quy tắc
A, B	INT8 có dấu	Bù 2. Không có dạng không dấu
$a \cdot b$	16 bit có dấu	Tính đủ 16 bit rồi **mở rộng dấu** lên 32 bit trước khi cộng
C (tích lũy)	INT32 có dấu	**Wrap** mod 2^{32}, không saturate. Trên miền hợp lệ không bao giờ tràn ($\|C\| \le 2^{19}$)
bias	INT32 có dấu	$acc = C + bias[j]$ trên INT32, **wrap** mod 2^{32}
M_0 (`QM0`)	INT32 có dấu	Mọi giá trị hợp lệ, kể cả -2^{31}
sh (`QSHIFT`)	5 bit không dấu	Miền 0 … 31
$prod$	64 bit có dấu	$acc \cdot M_0$; không tràn vì $\|prod\| \le 2^{62}$
y trước kẹp	64 bit có dấu	Dịch **số học**; làm tròn nửa lên (về $+\infty$)
Y	INT8 có dấu	**Saturate** về $[-128, 127]$. Đây là chỗ **duy nhất** trong thiết kế có saturation

QUYẾT ĐỊNH BASELINE D-27
Hai nguồn nêu rải rác và không hoàn toàn trùng cách diễn đạt về dấu và tràn; baseline gom thành một quy tắc duy nhất trong bảng 3.1: **mọi phép cộng số nguyên wrap, chỉ bước tạo Y saturate**. Hiện thực requantize trong VPU dùng hai bước tương đương biểu thức ở mục 3.1: $t = prod \ggg (30 + sh)$ giữ đủ **34 bit**, rồi $y = (t + 1) \ggg 1$ giữ đủ **33 bit**, rồi kẹp với $lo = \texttt{RELU_EN}\ ?\ 0 : -128$. Bắt buộc giữ đủ 34 và 33 bit: cắt t về 32 bit cho kết quả **sai** tại $acc = M_0 = -2^{31}$. Đây là một lỗi hiện thực đã được nhận diện trong nguồn và baseline nâng lên thành yêu cầu bắt buộc, kèm điểm kiểm chứng tương ứng trong spec của VPU.

3.3 Kích thước ma trận và căn chỉnh

Miền kích thước của một descriptor: $1 \le M \le 32$; $N, K \in \{4, 8, \ldots, 32\}$.

Bảng 3.2: Quy tắc kích thước và căn chỉnh

Hạng mục	Quy tắc
M	Bất kỳ giá trị 1 … 32. Không cần là bội của 4, không cần là bội của P
N, K	Bội của 4 trong miền 4 … 32. Ma trận có N hoặc K không chia hết cho 4 phải được **phần mềm** đệm 0 lên bội của 4; cột thừa của kết quả do phần mềm bỏ qua
Không chia hết cho P = 8	Xử lý **trong phần cứng**, không cần đệm: $TM = \lceil M/8 \rceil$, $TN = \lceil N/8 \rceil$; tile biên dùng `row_vld[r]` = (i0 + r < M) chặn hàng thừa và `o_row_mask` (8 bit) chặn cột thừa
K không chia hết cho 8	Không cần xử lý gì: K là chiều tích lũy, bộ tuần tự tile phát đúng K bước rồi chèn bubble tới $I = \max(K, 8)$
Địa chỉ cơ sở	Mọi địa chỉ cơ sở được dùng phải căn 4 byte. `DESC_PTR` và mọi `NEXT_PTR` được dùng phải căn 64 byte
Biên word	Vì N và K là bội của 4, mọi hàng bắt đầu và kết thúc đúng biên word, nên `m_axi_wstrb` luôn bằng 4'b1111
Dung lượng bank	A, B, bias và C của một descriptor nằm trọn trong một bank của buffer tương ứng: bank A và B sâu 256 word, C 1024 word, BIAS 32 word, đủ cho M, N, K $\le$ 32

QUYẾT ĐỊNH BASELINE D-26
Baseline chốt ranh giới trách nhiệm: **phần mềm** đệm tới bội của 4, **phần cứng** xử lý phần dư so với P = 8. Lý do: đệm tới bội 4 là điều kiện đủ để mọi hàng DRAM trùng biên word 32 bit, nhờ đó DMA không bao giờ cần `wstrb` từng phần; còn buộc phần mềm đệm tới bội 8 sẽ đòi thêm bộ nhớ và thêm bước xử lý mà phần cứng vốn đã có đủ cơ chế (`row_vld` và `o_row_mask`) để chặn. Hai cơ chế chặn khác nhau: **hàng** thừa có `row_vld[r]` = 0 nên PE của hàng đó không cập nhật `acc` và không phát `trig`, tức hàng đó không bao giờ được xả; **cột** thừa vẫn được PE tính bình thường nhưng kết quả không được ghi, vì `o_row_mask` đi tiếp thành `i_c_wr_mask` rồi thành byte/word enable của bank C.

3.4 Bố trí dữ liệu trong DRAM

Bảng 3.3: Địa chỉ byte của phần tử

Ma trận	Địa chỉ khi stride = 0	Kích thước vùng
A	`A_ADDR` + i·K + k	M·K byte
B	`B_ADDR` + k·N + j	K·N byte
bias	`BIAS_ADDR` + 4j	4·N byte
C (INT32)	`C_ADDR` + 4·(i·N + j)	4·M·N byte
Y (INT8)	`C_ADDR` + i·N + j	M·N byte

Bố trí dữ liệu

- Địa chỉ trong bảng là lúc stride bằng 0 (các hàng liền nhau). Khi `LDA`, `LDB`, `LDC` khác 0 thì dùng công thức ở mục 5.2
- Little-endian: byte tại địa chỉ a nằm ở bit [8·(a mod 4)+7 : 8·(a mod 4)] của word 32 bit; INT32 là little-endian
- Mọi địa chỉ cơ sở căn 4 byte. Vì N, K là bội của 4, mọi hàng bắt đầu và kết thúc đúng biên word, nên `m_axi_wstrb` luôn bằng 4'b1111
- Vùng đích của một descriptor không được chồng lên A, B hoặc bias của descriptor đó; A, B, bias được phép chồng nhau. Vùng đích của mọi descriptor không được chồng lên vùng descriptor. Vi phạm → kết quả không xác định. Phần cứng không kiểm tra
- Phần cứng không kiểm tra vùng nhớ có nằm trong DRAM không; bus trả lỗi thì tác vụ kết thúc với ERROR (mục 4.4)

QUYẾT ĐỊNH BASELINE D-24
Phần cứng **không** kiểm tra giới hạn địa chỉ DRAM và **không** kiểm tra chồng lấn vùng nhớ. Lý do: kiểm chồng lấn cần biết toàn bộ bản đồ bộ nhớ của SoC và cần so sánh mọi cặp vùng của cả chuỗi, không khả thi trong accelerator. Phần cứng chỉ kiểm những điều kiện cục bộ của descriptor ở mục 5.2 (căn chỉnh, miền M/N/K, stride, bit dự trữ). Tác động: vi phạm chồng lấn cho kết quả không xác định mà không báo lỗi; truy cập ngoài vùng DRAM sẽ do bus phát hiện và trở thành `ERR_CODE` = 2, 3 hoặc 4.

3.5 Mô hình tham chiếu

```
import numpy as np

def gemm(A, B):                        # A: int8 MxK, B: int8 KxN
    return (A.astype(np.int64) @ B.astype(np.int64)).astype(np.int32)

def post(C, bias, M0, sh, relu):       # C: int32 MxN, bias: int32 N
    acc  = (C.astype(np.int64) + bias).astype(np.int32).astype(np.int64)
    prod = acc * np.int64(M0)
    y    = (prod + (np.int64(1) << (30 + sh))) >> (31 + sh)
    if relu:
        y = np.maximum(y, 0)
    return np.clip(y, -128, 127).astype(np.int8)
```

Nội dung DRAM sau một tác vụ phải khớp từng bit với mô hình trên, áp dụng lần lượt cho từng descriptor theo đúng thứ tự chuỗi, bất kể các pha đã chạy chồng nhau đến mức nào.

3.6 Ma trận lớn hơn 32

Một descriptor giới hạn M, N, K $\leq$ 32. Lớp lớn hơn được phần mềm chia thành nhiều descriptor:

Bảng 3.4: Chia lớp lớn hơn 32

Chiều	Cách chia
M	Chia thành các dải 32 hàng. Mỗi dải là một descriptor độc lập; A, C dùng địa chỉ cơ sở lệch theo hàng và stride `LDA`, `LDC` bằng độ dài hàng thật
N	Chia thành các dải cột $\leq$ 32. Dải bắt đầu ở cột j_0 dùng đoạn bias tương ứng (`BIAS_ADDR` lệch $4j_0$ byte, tức 32 word = 128 byte mỗi dải 32 cột), `B_ADDR` lệch j_0 byte, `C_ADDR` lệch $4j_0$ byte (C INT32) hoặc j_0 byte (Y INT8); `LDB`, `LDC` bằng độ dài hàng thật; `REUSE_B` không dùng được giữa các dải cột khác nhau
K	Chia thành các đoạn $\leq$ 32 với `POST_EN` = 0, ghi ra các vùng C INT32 riêng; CPU cộng dồn các vùng đó rồi tự thực hiện bias, requantize, ReLU và kẹp. Không có chế độ $C \mathrel{+}= A \times B$ trong phần cứng

Nội dung mục này đến từ `spec_2` (REQ-AP-001); `spec_1` không có phần tương ứng. Baseline giữ lại vì nó cần thiết để phần mềm dùng được accelerator cho lớp mạng thật, và nó không mâu thuẫn với bất kỳ quyết định nào ở trên.

3.7 Thông lượng

Bảng 3.5: Thông lượng baseline, M = N = K = 32, `POST_EN` = 0, stride = 0

Hạng mục	Khối lượng (word, beat hoặc chu kỳ)	Thời gian ở 100 MHz
FETCH	16 word, 1 burst 16 beat	0,16 µs
LOAD_A	32 hàng $\times$ 8 word = 256 word	2,56 µs
LOAD_B	32 hàng $\times$ 8 word = 256 word	2,56 µs
STORE_C (INT32)	32 hàng $\times$ 32 word = 1024 word	10,24 µs
Tổng beat AXI	1552 beat	15,52 µs
COMPUTE	534 chu kỳ	5,34 µs
Đỉnh tính toán của array	64 MAC/chu kỳ	6,4 GMAC/s (12,8 GOP/s)

QUYẾT ĐỊNH BASELINE D-25
Hai nguồn đưa các số liệu thành phần nhưng không nguồn nào tổng hợp thành bức tranh thông lượng; baseline tính và chốt bảng 3.5 từ các số liệu đó. Kết luận dùng cho thiết kế: với M = N = K = 32 và `POST_EN` = 0, tác vụ **bị giới hạn bởi băng thông AXI 32 bit** chứ không bởi array (534 chu kỳ ≈ 5,34 µs): một descriptor chạy đơn lẻ cần 1552 beat ≈ 15,52 µs; khi chạy chồng, kênh đọc (528 beat) và kênh ghi (1024 beat) hoạt động song song nên khâu chậm nhất là STORE_C (≈ 10,24 µs). Vì vậy scratchpad ping-pong che được toàn bộ thời gian COMPUTE, và thời gian mỗi descriptor tiến tới $\max(\text{nạp}, \text{tính}, \text{ghi}) \approx$ thời gian STORE_C. Hệ quả thiết kế: không cần làm array nhanh hơn; cải thiện thông lượng phải đến từ độ rộng bus AXI hoặc từ `POST_EN` = 1 (STORE_C giảm bốn lần, còn 256 word). Với `POST_EN` = 1, tổng beat là 16 + 32 + 256 + 256 + 256 = 816, tức 8,16 µs, và COMPUTE (5,34 µs) bắt đầu trở thành đáng kể.

4 Luồng vận hành

Một tác vụ là một chuỗi descriptor. Ba engine chạy đồng thời khi bank cho phép: **Đọc** (FETCH, kiểm tra, LOAD_*), **Tính** (COMPUTE), **Ghi** (STORE_C). Trong một descriptor, các pha đi theo thứ tự FETCH → kiểm tra → LOAD_BIAS (nếu `POST_EN`) → LOAD_A → LOAD_B (trừ khi `REUSE_B`) → COMPUTE → STORE_C. Engine Đọc làm xong mọi LOAD của descriptor d rồi mới FETCH descriptor d+1.

4.1 Các pha của một descriptor

Bảng 4.1: Các pha của một descriptor

Pha	Khối chính	Làm gì	Xong khi
FETCH	DMA	Đọc một descriptor, 16 word, một burst INCR 16 beat. Descriptor không vào scratchpad	FSM nhận đủ 16 word, hoặc lỗi mã 4
Kiểm tra	Main FSM	Kiểm tra descriptor theo mục 5.2. Sai: lỗi mã 1	Đúng một chu kỳ
LOAD_BIAS	DMA	Chép N word bias từ DRAM vào một bank BIAS. Chỉ có khi `POST_EN` = 1	SRAM nhận word cuối
LOAD_A	DMA	Chép A (M hàng × K/4 word) vào một bank A	SRAM nhận word cuối
LOAD_B	DMA	Chép B (K hàng × N/4 word) vào một bank B. Bỏ qua khi `REUSE_B` = 1	SRAM nhận word cuối
COMPUTE	Array, VPU	Array tính mọi tile; mỗi hàng kết quả đi qua VPU (hậu xử lý hoặc đi thẳng) rồi ghi vào bank C	`o_pe_done`
STORE_C	DMA	Chép C (M·N word) hoặc Y (M·N/4 word) từ bank C ra DRAM	Phản hồi B cuối là OKAY

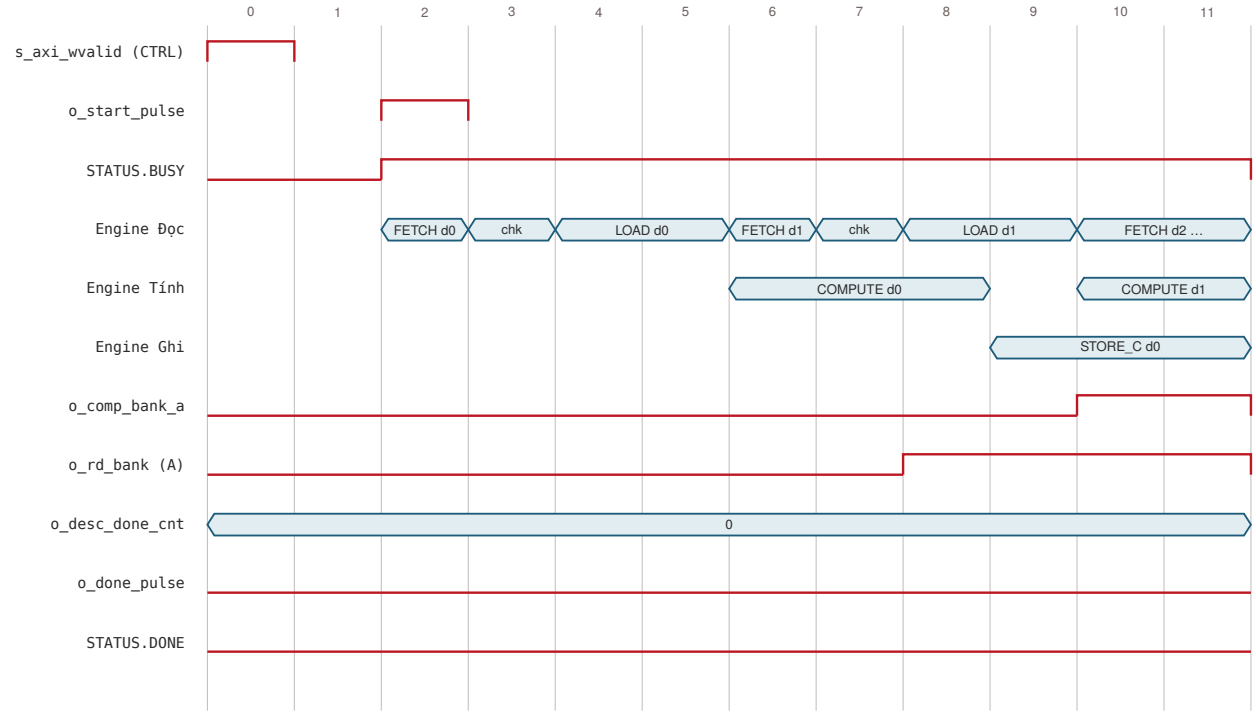
Bảng 4.2: Lệnh DMA của từng pha

Pha	Kênh	dst / fmt	rows	rwords	stride	base
FETCH	Đọc	dst = 3	1	16	64	Con trỏ descriptor
LOAD_BIAS	Đọc	dst = 2	1	N	4N	`BIAS_ADDR`
LOAD_A	Đọc	dst = 0	M	K/4	`LDA`	`A_ADDR`
LOAD_B	Đọc	dst = 1	K	N/4	`LDB`	`B_ADDR`
STORE_C	Ghi	fmt = `POST_EN`	M	`POST_EN` ? N/4 : N	`LDC`	`C_ADDR`

`rows` và `rwords` rộng 6 bit, `stride` rộng 16 bit, tính theo **word** cho `rwords` và theo **byte** cho `stride`. Khi stride bằng 0, Main FSM phát stride bằng đúng độ dài hàng (K cho A, N cho B, 4N cho C INT32, N cho Y). Bias là vector INT32 nên `rwords` = N word và độ dài hàng là 4N byte.

4.2 Luồng một tác vụ

Có ba mức ”xong”: `o_rd_done` và `o_wr_done` của DMA báo xong một lệnh DMA; `o_pe_done` báo toàn bộ kết quả của job đã nằm trong bank C; `STATUS`.DONE báo descriptor có `LAST` = 1 đã ghi xong DRAM và không có lỗi. Chỉ mức cuối cùng hiện ra cho CPU.



Hình 4.1: Hai descriptor đầu của một chuỗi. Ba engine chạy chồng: trong lúc engine Tính xử lý descriptor d0, engine Đọc FETCH, kiểm tra và nạp d1 vào bank kia; khi COMPUTE d0 xong, engine Ghi chép kết quả d0 ra DRAM trong lúc engine Tính xử lý d1. Trục thời gian là trục pha, không theo tỉ lệ số chu kỳ thực (riêng `o_start_pulse` ở chu kỳ 2 theo hình 6-1 của tài liệu Register File). d0 dùng bank A 0 nên `o_comp_bank_a` = 0 suốt COMPUTE d0, trong khi LOAD d1 ghi bank 1 (`o_rd_bank` = 1): hai phía luôn ở hai bank khác nhau. `o_rd_bank` không có nghĩa trong FETCH vì descriptor không vào scratchpad (D-31)

Ba engine được phép làm ba job khác nhau trong cùng một chu kỳ. Phần cứng được phép bắt đầu một pha muộn hơn điều kiện ở mục 5.3, không bao giờ sớm hơn. Khi các pha chồng nhau, `PERF_LOAD` + `PERF_COMPUTE` + `PERF_STORE` có thể lớn hơn `PERF_TOTAL`; phần chênh là thời gian chạy chồng.

4.3 Quyền truy cập scratchpad

Bảng 4.3: Ai truy cập buffer trong pha của một job

Pha	A	B	BIAS	C
FETCH, kiểm tra	—	—	—	—
LOAD_BIAS	—	—	DMA ghi	—
LOAD_A	DMA ghi	—	—	—
LOAD_B	—	DMA ghi	—	—
COMPUTE	Array đọc	Array đọc	VPU đọc	VPU ghi
STORE_C	—	—	—	DMA đọc

Quyền truy cập

- Bảng trên là quyền của một job. Job khác có thể đang ở pha khác, trên bank kia
- Mỗi bank có tối đa một bên truy cập trong một chu kỳ. Hai bank của cùng một buffer truy cập độc lập. Vì

vậy mỗi bank chỉ cần SRAM một cổng

- Điều kiện được bắt đầu một pha nằm ở mục 5.3
- Module SRAM có assertion (chỉ mô phỏng) phát hiện hai phía chạm cùng một sub-bank trong khi `i_comp_active` = 1. Trong RTL, khi trùng thì DMA thắng, nên hành vi luôn xác định (quyết định baseline D-12)

4.4 Xử lý lỗi

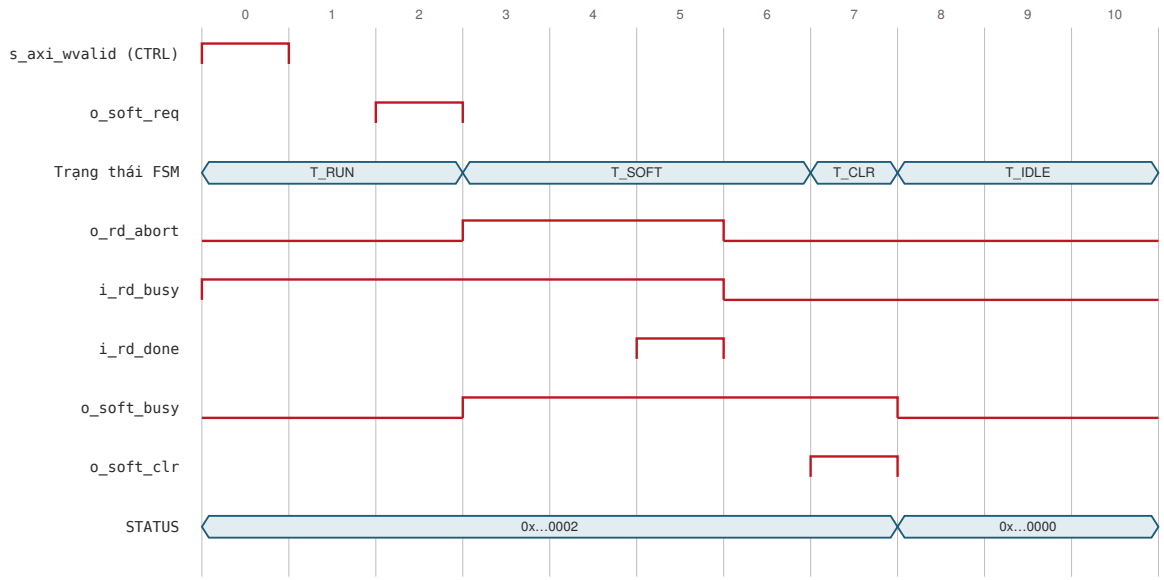
Lỗi

- `DESC_PTR` không căn 64 byte: `ERR_CODE` = 1, không truy cập bus, không truy cập scratchpad
- Descriptor sai sau FETCH: `ERR_CODE` = 1. Không pha nào tiếp của job đó được bắt đầu
- Lỗi AXI của FETCH: `ERR_CODE` = 4. Lỗi AXI của LOAD_*: mã 2. Lỗi AXI của STORE_C: mã 3. DMA của kênh lỗi ngừng phát burst mới, nhận đủ beat R hoặc phản hồi B của burst đã phát, rồi báo lỗi
- Gọi f là descriptor có thứ tự nhỏ nhất bị lỗi. Không pha nào của descriptor thứ $\geq$ f được bắt đầu. Pha đã bắt đầu thì chạy tới hết. Các descriptor đứng trước f chạy xong, kể cả STORE_C. Khi mọi engine đã yên: ERROR = 1, `ERR_DESC_ADDR` = địa chỉ của f, `DESC_DONE_CNT` = f. Vùng đích của job $\geq$ f không xác định
- Khi nhiều lỗi xảy ra, Main FSM giữ lỗi có **chỉ số descriptor nhỏ nhất**; nếu cùng chỉ số thì giữ lỗi ghi nhận trước. Chỉ một `ERR_CODE` được báo cho CPU
- Array và VPU không có đầu ra lỗi. Local SRAM không có đầu ra lỗi. Mọi mã lỗi đều do Main FSM phát, từ kết quả kiểm tra descriptor hoặc từ `o_rd_error` / `o_wr_error` của DMA
- Beat dữ liệu của một burst lỗi vẫn được ghi vào scratchpad; nội dung đó không được dùng vì không pha nào tiếp theo được bắt đầu

4.5 Reset và soft reset

Reset

- `rst_n` tích cực mức thấp, bất đồng bộ. `gemm_accel_top` có bộ đồng bộ 2 flip-flop (bật bất đồng bộ, nhả đồng bộ với `clk`); đầu ra `rst_n_sync` cấp cho cả sáu module
- `rst_n_sync` = 0 đưa mọi trạng thái về giá trị reset. Nội dung scratchpad không bị xóa, coi là không xác định
- Soft reset không đi vào chân reset; nó là xung xóa đồng bộ `soft_clr` một chu kỳ do Main FSM phát tới Register File, DMA, SRAM, Array, VPU. Main FSM tự reset trạng thái của nó ở cùng chu kỳ và không có port `i_soft_clr`
- Phạm vi soft reset: FSM, DMA, Array, VPU, phần điều khiển SRAM; `STATUS`, `PERF_*`, `ERR_DESC_ADDR` và `DESC_DONE_CNT`. `DESC_PTR` và các thanh ghi RW giữ nguyên. Nội dung scratchpad không xác định
- Khi có yêu cầu soft reset: không pha mới nào bắt đầu; không phát lệnh DMA mới. FSM abort kênh đọc nếu kênh đó đang bận và kênh ghi nếu kênh đó đang bận. Burst đã cam kết (đã bật VALID hoặc đã gửi beat W) chạy hết; lệnh kết thúc bằng done hoặc error. Abort không phải là lỗi của tác vụ
- `soft_clr` phát khi cả hai kênh DMA đã rảnh (`i_rd_busy` = 0 và `i_wr_busy` = 0). COMPUTE đang chạy không được chờ: `soft_clr` hủy array và VPU ngay
- Nếu yêu cầu soft reset đến đúng chu kỳ một kênh DMA báo xong, kênh đó coi như đã rảnh
- Trong cả chuỗi soft reset: BUSY = 1 và `STATUS` đọc ra 0x0000_0002; START và SOFT_RESET ghi lặp lại bị bỏ qua
- Sau `soft_clr`: `STATUS` = 0; tác vụ bị hủy không báo DONE cũng không báo ERROR; vùng đích trong DRAM của tác vụ bị hủy không xác định



Hình 4.2: Chuỗi soft reset. Bắt tay AW, W của lệnh ghi `CTRL` có SOFT_RESET = 1 ở chu kỳ 0; lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ 1 (`wr_fire`); Register File phát `o_soft_req` ở chu kỳ 2 (cùng mốc với `o_start_pulse` trong hình 6-1 của tài liệu Register File); FSM vào `T_SOFT` và abort kênh đọc đang bận; burst đã cam kết chạy hết và báo `i_rd_done` ở chu kỳ 5; khi không engine nào còn chờ lệnh DMA, FSM vào `T_CLR` và phát `o_soft_clr` một chu kỳ, rồi về `T_IDLE` với `STATUS` = 0. Không có DONE và không có ERROR

5 Chuỗi descriptor và scratchpad ping-pong

Kiến trúc chỉ có một chế độ. CPU dựng danh sách descriptor trong DRAM và ra một lệnh START cho cả chuỗi. Mỗi buffer A, B, C, BIAS có hai bank. Nạp, tính và ghi của các descriptor kế nhau chạy chồng trên hai bank.

5.1 Nguyên tắc

Nguyên tắc

- Không thêm, bớt hay đổi port top-level (mục 1.7). Offset thanh ghi giữ nguyên; 0x08 … 0x24 dự trữ vì tham số của job nằm trong descriptor
- DMA có kênh đọc và kênh ghi độc lập, có lệnh FETCH riêng. SRAM có bit chọn bank cho từng buffer. Main FSM lập lịch ba engine
- Tham số của mỗi job lấy từ descriptor. `DESC_PTR` là bản chụp duy nhất lúc START

5.2 DMA theo chuỗi descriptor

CPU dựng sẵn một danh sách descriptor trong DRAM, mỗi descriptor mô tả một phép GEMM. CPU ghi địa chỉ descriptor đầu vào `DESC_PTR`, thực hiện `fencerw,rw`, ghi `CTRL[2]` = 1 (START), rồi chờ DONE một lần cho cả chuỗi. Accelerator tự đọc từng descriptor và chạy hết chuỗi.

Bảng 5.1: Bố cục descriptor (16 word, 64 byte)

Offset	Word	Tên	Bit	Nội dung
0x00	0	`DIM`	[31:0]	M[7:0], N[15:8], K[23:16]; [31:24] phải bằng 0
0x04	1	`A_ADDR`	[31:0]	Địa chỉ byte của A[0][0]
0x08	2	`B_ADDR`	[31:0]	Địa chỉ byte của B[0][0]; bỏ qua khi `REUSE_B` = 1
0x0C	3	`C_ADDR`	[31:0]	Địa chỉ byte của C[0][0] hoặc Y[0][0]

Bảng 5.1: Bố cục descriptor (16 word, 64 byte) (tiếp theo)

Offset	Word	Tên	Bit	Nội dung
0x10	4	LDA, LDB	[15:0], [31:16]	Stride theo hàng của A và B (byte); 0 = các hàng liền nhau
0x14	5	LDC	[15:0]	Stride của C hoặc Y (byte); 0 = liền nhau; [31:16] dự trữ
0x18	6	FLAGS	[4:0]	[0] LAST, [1] WAIT_PREV, [2] REUSE_B, [3] POST_EN, [4] RELU_EN; [31:5] dự trữ
0x1C	7	NEXT_PTR	[31:0]	Địa chỉ descriptor kế tiếp, căn 64 byte; bỏ qua khi LAST = 1
0x20	8	BIAS_ADDR	[31:0]	Địa chỉ byte của bias[0]; chỉ dùng khi POST_EN = 1
0x24	9	QM0	[31:0]	Hệ số nhân M_0, INT32 có dấu; chỉ dùng khi POST_EN = 1
0x28	10	QSHIFT	[4:0]	Lượng dịch sh, 0 … 31; [31:5] dự trữ
0x2C	11	TAG	[31:0]	Tùy phần mềm; phần cứng không đọc, mọi giá trị hợp lệ
0x30–0x3C	12–15	—	—	Dự trữ, phải bằng 0

Với stride, địa chỉ phần tử là: A[i][k] tại A_ADDR + i·LDA + k; B[k][j] tại B_ADDR + k·LDB + j; C[i][j] tại C_ADDR + i·LDC + 4j; Y[i][j] tại C_ADDR + i·LDC + j. Stride = 0 được hiểu là đúng độ dài hàng.

NGUỒN

Bố cục 16 word ở bảng 5.1 chỉ có trong spec_1. spec_2 xác nhận descriptor dài 16 word và 64 byte, xác nhận lệnh FETCH (rows = 1, rwords = 16, stride = 64), xác nhận yêu cầu căn 64 byte và xác nhận các cờ LAST, WAIT_PREV, REUSE_B cùng word TAG bị bỏ qua, nhưng **không** nêu vị trí từng word và từng bit. Vì vậy baseline lấy nguyên bố cục của spec_1; không có mâu thuẫn cần giải, chỉ có một nguồn chi tiết hơn.

5.2.1 Lấy và tiêu thụ descriptor

Bảng 5.2: Lấy và tiêu thụ một descriptor

Bước	Nội dung
Phát lệnh FETCH	Main FSM phát lệnh kênh đọc với o_rd_dst = 3, o_rd_base = con trỏ descriptor, o_rd_rows = 1, o_rd_rwords = 16, o_rd_stride = 64, o_rd_bank = 0
Truyền AXI	Một burst INCR 16 beat, m_axi_arlen = 15, m_axi_arsize = 3'b010. Burst này không bao giờ vượt biên 4 KiB vì địa chỉ căn 64 byte và độ dài 64 byte
Trả về	Descriptor **không** đi vào scratchpad. DMA trả từng word qua o_desc_valid, o_desc_idx[3:0] = 0 … 15, o_desc_data[31:0]
Nhận	Main FSM ghi 16 word vào bộ đệm descriptor nội bộ. Xong khi nhận đủ word thứ 15, hoặc lỗi mã 4
Kiểm tra	Một chu kỳ: gemm_job_check dựng job_t và kiểm điều kiện hợp lệ ở mục 5.2.2. Sai → ERR_CODE = 1
Con trỏ kế tiếp	Lấy từ NEXT_PTR của descriptor vừa kiểm. LAST = 1 kết thúc chuỗi, NEXT_PTR bị bỏ qua
Thứ tự	Mỗi descriptor được đọc **đúng một lần**, theo đúng thứ tự chuỗi. Engine Đọc làm tuần tự FETCH(d) → kiểm tra(d) → mọi LOAD của d → FETCH(d+1)

5.2.2 Điều kiện descriptor hợp lệ

Descriptor hợp lệ khi **tất cả** các điều kiện sau đúng. Sai bất kỳ điều kiện → ERR_CODE = 1 và descriptor đó là descriptor lỗi.

Bảng 5.3: Điều kiện descriptor hợp lệ

Hạng mục	Điều kiện
Kích thước	$1 \le M \le 32$; N, K $\in$ {4, 8, …, 32}
Bit dự trữ của DIM	DIM[31:24] = 0

Bảng 5.3: Điều kiện descriptor hợp lệ (tiếp theo)

Hạng mục	Điều kiện
Căn chỉnh địa chỉ	Mọi địa chỉ **được dùng** căn 4 byte: A_ADDR, C_ADDR, B_ADDR (trừ khi REUSE_B = 1), BIAS_ADDR (chỉ khi POST_EN = 1)
Con trỏ kế tiếp	NEXT_PTR căn 64 byte khi LAST = 0
Stride	Mỗi stride (LDA, LDB, LDC) bằng 0, hoặc đồng thời ≥ độ dài hàng và là bội của 4. Độ dài hàng: K cho A, N cho B, 4N cho C khi POST_EN = 0, N cho Y khi POST_EN = 1. Khác với địa chỉ, cả ba stride luôn bị kiểm, kể cả LDB khi REUSE_B = 1
Bit và word dự trữ	FLAGS[31:5] = 0; LDC[31:16] = 0; QSHIFT[31:5] = 0; word 12 … 15 = 0
Ràng buộc cờ	RELU_EN = 1 kéo theo POST_EN = 1
Dùng lại B	REUSE_B = 1 chỉ khi đây không phải descriptor đầu chuỗi **và** K, N bằng của descriptor ngay trước

Word TAG không bị kiểm: mọi giá trị đều hợp lệ và phần cứng không đọc nó.

Chế độ chuỗi

- DESC_PTR và mọi NEXT_PTR được dùng phải căn 64 byte; nếu không, ERR_CODE = 1
- Mỗi descriptor chạy các pha FETCH → kiểm tra → LOAD_BIAS (nếu POST_EN) → LOAD_A → LOAD_B (trừ khi REUSE_B) → COMPUTE → STORE_C, với cùng phép toán và cùng bố trí DRAM ở mục 3
- REUSE_B = 1: không nạp B; COMPUTE dùng B của descriptor ngay trước, trên đúng bank B của descriptor đó
- WAIT_PREV = 1: các pha LOAD của descriptor chỉ bắt đầu khi STORE_C của mọi descriptor trước nó đã xong, tức khi DESC_DONE_CNT bằng chỉ số của descriptor này. Phần mềm phải đặt cờ này khi descriptor đọc dữ liệu do descriptor trước ghi ra. WAIT_PREV chặn các LOAD, không chặn FETCH và kiểm tra
- Chuỗi kết thúc ở descriptor đầu tiên có LAST = 1. DONE = 1 khi STORE_C của descriptor đó nhận phản hồi B cuối là OKAY
- DESC_DONE_CNT tăng 1 mỗi khi STORE_C của một descriptor xong OKAY; các descriptor xong đúng theo thứ tự chuỗi
- Lỗi: gọi f là descriptor có thứ tự nhỏ nhất bị lỗi. Không pha nào của descriptor thứ ≥ f được bắt đầu; các descriptor đứng trước f được chạy xong. Khi mọi khối đã rảnh: ERROR = 1, ERR_CODE theo bảng 2.12 (lỗi FETCH là mã 4), ERR_DESC_ADDR = địa chỉ của f, DESC_DONE_CNT = số thứ tự của f
- Khi dùng stride lớn hơn độ dài hàng, DMA cắt burst theo từng hàng: burst không bao giờ trải qua khoảng hở giữa hai hàng, và các byte trong khoảng hở không bị đọc hay ghi
- Accelerator không ghi vào descriptor; phần mềm không sửa descriptor và vùng dữ liệu của chuỗi đang chạy. Vùng đích của mọi descriptor không được chồng lên vùng descriptor
- Tham số hậu xử lý của job lấy từ descriptor: POST_EN, RELU_EN, BIAS_ADDR, QM0, QSHIFT

5.3 Scratchpad ping-pong

Mỗi buffer A, B, C, BIAS có hai bank. DMA kênh đọc nạp descriptor d+1 vào một bank trong lúc array tính descriptor d trên bank kia, và DMA kênh ghi chép kết quả của descriptor d−1 ra DRAM. Thời gian mỗi descriptor tiến tới $\max(\text{nạp}, \text{tính}, \text{ghi})$ thay vì tổng của chúng.

Bảng 5.4: Chọn bank cho descriptor d

Buffer	Quy tắc
A, C	Đảo bank sau mỗi descriptor
B	Đảo sau mỗi lần có LOAD_B; với REUSE_B = 1, dùng lại bank của descriptor trước
BIAS	Đảo sau mỗi descriptor có POST_EN = 1

Bảng 5.4: Chọn bank cho descriptor d (tiếp theo)

Buffer	Quy tắc

5.3.1 Đồng bộ producer và consumer

Main FSM giữ một *scoreboard* đếm số bên còn sẽ đọc mỗi bank. Đây là cơ chế duy nhất bảo đảm không có bên nào đọc một bank đang bị ghi, hoặc ghi một bank đang bị đọc.

Bảng 5.5: Scoreboard bank trong Main FSM

Bộ đếm	Độ rộng	Ý nghĩa và cập nhật
`rcnt[A][b]`, `rcnt[B][b]`, `rcnt[BIAS][b]`	2 bit	Số job còn sẽ **đọc** bank b của buffer đó (phía COMPUTE). Tăng ở chu kỳ engine Đọc bắt đầu LOAD_X của job vào bank b, hoặc ở bước R_RB với job `REUSE_B` (không tăng lúc gán bank ở CHECK, nếu không LOAD_X sẽ chờ chính bộ đếm vừa tăng); giảm khi `i_pe_done` của job đó. LOAD_X vào bank b chỉ được bắt đầu khi `rcnt[X][b]` = 0
`wcnt[C][b]`	1 bit	Số job còn sẽ **đọc** bank C b (phía STORE_C). Tăng khi COMPUTE của job có bank C = b bắt đầu; giảm khi STORE_C của job đó xong OKAY (`i_wr_done`). COMPUTE ghi bank C b chỉ được bắt đầu khi `wcnt[C][b]` = 0

Trọng tài giữa ba engine khi cùng muốn phát lệnh DMA hoặc cùng muốn tiến: ưu tiên cố định **Ghi > Tính > Đọc**. Lý do của thứ tự này: engine Ghi giải phóng bank C, engine Tính giải phóng bank A, B, BIAS; ưu tiên khâu giải phóng tài nguyên trước làm engine Đọc không bị khóa lâu và giữ đường ống chạy đầy.

Ping-pong

- Ba khối thực thi: Đọc (FETCH, kiểm tra, LOAD_*; kênh đọc DMA), Tính (COMPUTE; array và VPU), Ghi (STORE_C; kênh ghi DMA). Mỗi khối làm một việc một lúc, theo thứ tự descriptor. Kênh đọc và kênh ghi được phép có burst AXI cùng lúc, mỗi kênh tối đa 1 burst outstanding
- Khối Đọc làm lần lượt FETCH(d), kiểm tra d, các LOAD của d, rồi mới sang FETCH(d+1)
- LOAD vào một bank chỉ bắt đầu khi COMPUTE cuối cùng đọc bank đó đã xong
- COMPUTE(d) bắt đầu khi mọi LOAD của d và COMPUTE(d−1) đã xong, và STORE_C cuối cùng đọc bank C của d đã xong
- STORE_C(d) bắt đầu khi COMPUTE(d) và STORE_C(d−1) đã xong; do đó kết quả ra DRAM đúng theo thứ tự descriptor
- `WAIT_PREV` chặn các LOAD, không chặn FETCH và kiểm tra
- Tại mỗi thời điểm, mỗi bank có tối đa một bên truy cập. Hai bank khác nhau của cùng buffer được truy cập độc lập trong cùng chu kỳ; mỗi bank vẫn chỉ cần SRAM một cổng
- Các điều kiện thứ tự và quyền truy cập bank nêu trên là điều kiện **cần**; phần cứng được phép bắt đầu một pha muộn hơn, không bao giờ sớm hơn. Nội dung DRAM sau tác vụ phải giống hệt thực thi tuần tự từng descriptor
- Soft reset: abort cả hai kênh DMA đang bận; `soft_clr` phát khi cả hai kênh DMA đã rảnh (`i_rd_busy` = 0 và `i_wr_busy` = 0)
- `PERF_LOAD` + `PERF_COMPUTE` + `PERF_STORE` được phép lớn hơn `PERF_TOTAL`; phần chênh là thời gian chạy chồng

5.3.2 Hành vi biên của ping-pong

Bảng 5.6: Các trường hợp biên

Trường hợp	Hành vi
Chuỗi một descriptor	Ba engine chạy tuần tự; không có gì để chồng. Bank dùng là bank 0 của mọi buffer

Bảng 5.6: Các trường hợp biên (tiếp theo)

Trường hợp	Hành vi
Descriptor đầu chuỗi	`rcnt` và `wcnt` đã được xóa ở chu kỳ START được chấp nhận, nên không engine nào bị chặn
`REUSE_B` liên tiếp	Bank B không đảo; `rcnt[B][b]` của bank đang dùng tăng thêm cho mỗi job dùng lại. Không có logic dừng riêng cho bộ đếm: theo cấu trúc pipeline, tại mọi thời điểm tối đa 3 job giữ một bank B (một job đang COMPUTE chưa có `i_pe_done`, một job trong slot, một job ở R_RB). Job ở C_HAND đã giảm `rcnt` lúc `i_pe_done`; job mới chỉ vào slot khi slot trống, tức sau `i_pe_done` của job trước. Vì vậy `rcnt` 2 bit (tối đa 3) không bao giờ tràn; AS-03 của Main FSM kiểm điều này
`POST_EN` xen kẽ	Bank BIAS chỉ đảo ở descriptor có `POST_EN` = 1; descriptor có `POST_EN` = 0 không nạp bias, không đảo bank BIAS và không tăng `rcnt[BIAS]`
`WAIT_PREV` = 1	Engine Đọc đã FETCH và kiểm tra xong nhưng dừng trước LOAD; engine Tính và Ghi vẫn chạy descriptor trước. Ping-pong tạm thu về thực thi tuần tự ở ranh giới đó
Lỗi giữa chuỗi	Engine đã bắt đầu pha thì chạy tới hết; không pha mới nào của descriptor $\geq$ f bắt đầu. Các bộ đếm scoreboard vẫn giảm đúng khi các pha đang chạy kết thúc, nên hệ thống về trạng thái yên và ERROR được đặt
Soft reset giữa chuỗi	Không pha mới; abort kênh DMA đang bận; COMPUTE bị hủy ngay. Scoreboard bị xóa bởi `soft_clr`; nội dung scratchpad không xác định

6 Tích hợp các module

Bản chụp lúc START chỉ gồm `DESC_PTR` và chỉ đi vào Main FSM. FSM dựng job từ descriptor, cấp cấu hình của job đang tính cho array và VPU, cấp bank cho SRAM, và cấp địa chỉ, số hàng, số word, stride cùng bank trong từng lệnh DMA.

6.1 Kết nối giữa các module

Bảng 6.1: Kết nối giữa các module

Từ	Tới	Tín hiệu
RF `o_start_pulse`, `o_soft_req`, `o_cfg_desc_ptr`	FSM `i_start_pulse`, `i_soft_req`, `i_cfg_desc_ptr`	Xung start, xung soft reset và `DESC_PTR`
FSM `o_done_pulse`, `o_err_pulse`, `o_err_code`, `o_perf_load`, `o_perf_comp`, `o_perf_store`, `o_soft_busy`, `o_err_desc_addr`, `o_desc_done_cnt`	RF `i_*` cùng tên	Trạng thái, mã lỗi, bộ đếm hiệu năng
FSM `o_soft_clr`	`i_soft_clr` của RF, DMA, SRAM, Array, VPU	Xung xóa đồng bộ một chu kỳ; FSM không có `i_soft_clr`
FSM `o_rd_start`, `o_rd_dst`, `o_rd_base`, `o_rd_rows`, `o_rd_rwords`, `o_rd_stride`, `o_rd_bank`, `o_rd_abort`	DMA `i_rd_*` cùng tên	Lệnh kênh đọc
FSM `o_wr_start`, `o_wr_fmt`, `o_wr_base`, `o_wr_rows`, `o_wr_rwords`, `o_wr_stride`, `o_wr_bank`, `o_wr_abort`	DMA `i_wr_*` cùng tên	Lệnh kênh ghi
DMA `o_rd_busy`, `o_rd_done`, `o_rd_error`, `o_desc_valid`, `o_desc_idx`, `o_desc_data`, `o_wr_busy`, `o_wr_done`, `o_wr_error`	FSM `i_*` cùng tên	Trạng thái hai kênh và 16 word descriptor

Bảng 6.1: Kết nối giữa các module (tiếp theo)

Từ	Tới	Tín hiệu
DMA o_spm_wr_valid, o_spm_wr_buf, o_spm_wr_bank, o_spm_wr_addr, o_spm_wr_data	SRAM i_dw_valid, i_dw_buf, i_dw_bank, i_dw_addr, i_dw_data	Word dữ liệu, địa chỉ, buffer, bank
DMA o_spm_rd_valid, o_spm_rd_bank, o_spm_rd_addr	SRAM i_dr_valid, i_dr_bank, i_dr_addr	Yêu cầu đọc bank C
SRAM o_dw_ready, o_dr_ready, o_dr_rvalid, o_dr_rdata	DMA i_spm_wr_ready, i_spm_rd_ready, i_spm_rd_rvalid, i_spm_rd_rdata	o_dw_ready và o_dr_ready là hằng 1; dữ liệu về sau đúng một chu kỳ
FSM o_pe_start, o_comp_dim	Array i_pe_start, i_matrix_dim	Lệnh bắt đầu COMPUTE và {8'h00, K, N, M}
Array o_pe_done, o_busy	FSM i_pe_done, i_pe_busy	
FSM o_comp_post_en, o_comp_relu_en, o_comp_qm0, o_comp_qshift	VPU i_post_en, i_relu_en, i_quant_m0, i_quant_shift	Cấu hình hậu xử lý của job đang tính
FSM o_comp_bank_a, o_comp_bank_b, o_comp_bank_bias, o_comp_bank_c, o_comp_active	SRAM i_comp_bank_a, i_comp_bank_b, i_comp_bank_bias, i_comp_bank_c, i_comp_active	Bank cấp cho phía COMPUTE
Array o_a_rd_en, o_a_rd_row, o_a_rd_k, o_b_rd_en, o_b_rd_k, o_b_rd_col	SRAM i_a_rd_*, i_b_rd_*; về Array o_a_rd_data, o_b_rd_data	Đọc vector 64 bit; dữ liệu trả về trễ đúng một chu kỳ. o_a_rd_en luôn bằng o_b_rd_en
Array o_row_valid, o_row_idx, o_row_col, o_row_data, o_row_mask	VPU i_row_valid, i_row_idx, i_row_col, i_row_data, i_row_mask	Luồng hàng 256 bit; **không** có đường ready
VPU o_bias_rd_en, o_bias_rd_col, o_c_wr_en, o_c_wr_fmt, o_c_wr_row, o_c_wr_col, o_c_wr_data, o_c_wr_mask	SRAM i_bias_rd_*, i_c_wr_*; về VPU o_bias_rd_data	Đọc bias 256 bit, ghi bank C 256 bit có mask
top s_axi_*	RF	17 tín hiệu, đi thẳng, không logic chèn giữa
DMA m_axi_*	top	25 tín hiệu, đi thẳng, không logic chèn giữa

6.2 Quy tắc tích hợp

Tích hợp

- Tham số P phải thống nhất giữa Array, VPU, SRAM và trường P của HW_CONFIG; baseline chốt P = 8 cho cả bốn chỗ
- L_VPU của Array phải bằng LAT của VPU, cả hai lấy từ gemm_pkg::VPU_LATENCY = 4. Mỗi module kiểm điều kiện này lúc elaboration và phát \$error nếu sai
- Top truyền giá trị tham số đã chọn xuống các module, không phụ thuộc mặc định riêng của từng module
- Top chỉ đấu nối và chứa bộ đồng bộ reset 2 flip-flop; không có logic dữ liệu, không có FSM, độ trễ xuyên qua top là 0 chu kỳ
- rst_n_sync là reset duy nhất cấp cho cả sáu module. soft_clr đi tới năm module (RF, DMA, SRAM, Array, VPU) và không đi vào chân reset của bất kỳ flip-flop nào
- Thiết kế có đúng ba nhóm đường tổ hợp từ input tới output của module: (1) VPU: o_bias_rd_en = i_row_valid & i_post_en và o_bias_rd_col = i_row_col; (2) DMA: m_axi_rready = trạng thái RD_DATA ∧ i_spm_wr_ready, và vì o_dw_ready là hằng 1 nên m_axi_rready không phụ thuộc input AXI; (3) DMA: o_spm_wr_valid, o_spm_wr_data và (với FETCH) o_desc_valid, o_desc_data tổ hợp từ m_axi_rvalid, m_axi_rdata (đi vào SRAM hoặc bộ đệm descriptor của Main FSM, kết thúc ở thanh ghi/RAM). Ràng buộc I/O và kiểm tra vòng tổ hợp trong STA/lint phải bao phủ cả ba. Ready của Register File (s_axi_awready, s_axi_wready, s_axi_arready) chỉ phụ thuộc thanh ghi nội bộ, không phụ thuộc tổ hợp vào input. Mọi output khác của mọi module đều đã qua thanh ghi
- Không có đường tổ hợp từ input tới output trên cùng một cổng AXI

6.3 Quy tắc giao tiếp

Bảng 6.2: Quy ước bắt tay giữa các module

Loại giao tiếp	Quy ước
Xung một chu kỳ	`o_start_pulse`, `o_soft_req`, `o_soft_clr`, `o_done_pulse`, `o_err_pulse`, `o_rd_done`, `o_rd_error`, `o_wr_done`, `o_wr_error`, `o_pe_done`, `o_pe_start`: bằng 1 đúng một chu kỳ, bên nhận phải bắt ngay ở chu kỳ đó
Lệnh start	`o_rd_start` và `o_wr_start` là xung một chu kỳ; payload lệnh (`base`, `rows`, `rwords`, `stride`, `bank`, `dst/fmt`) phải hợp lệ ***ở đúng chu kỳ đó***. Start khi kênh đang bận bị bỏ qua
Mức (level)	`o_rd_abort`, `o_wr_abort`, `o_comp_active`, `o_comp_bank_*`, `o_soft_busy`, `i_rd_busy`, `i_wr_busy`, `i_pe_busy`: tín hiệu mức, giữ ổn định suốt khoảng có hiệu lực
Cấu hình job	`o_comp_dim`, `o_comp_post_en`, `o_comp_relu_en`, `o_comp_qm0`, `o_comp_qshift`: giữ ổn định từ trước `o_pe_start` tới khi `o_pe_done`. Assertion của VPU kiểm `\$stable` trong khi đường ống còn bit valid
Valid không ready	Luồng hàng Array → VPU (`o_row_valid` …) và luồng descriptor DMA → FSM (`o_desc_valid` …) không có đường ready: bên nhận **bắt buộc** nhận mỗi chu kỳ có valid. Điều này an toàn vì VPU có thông lượng 1 hàng/chu kỳ và bộ đệm descriptor của FSM luôn trống khi FETCH chạy
Valid có ready	Đường DMA ↔ SRAM dùng valid/ready, nhưng `o_dw_ready` và `o_dr_ready` là hằng 1, nên không bao giờ có backpressure. Hiện thực vẫn phải giữ đúng ngữ nghĩa valid/ready
Đọc SRAM	Mọi cổng đọc SRAM có độ trễ **đúng 1 chu kỳ**: dữ liệu hợp lệ ở chu kỳ ngay sau chu kỳ có `*_rd_en`. Không có đường tổ hợp từ `*_rd_en` tới dữ liệu

6.4 Xử lý lỗi giữa các module

Bảng 6.3: Nguồn lỗi và đường báo lỗi

Khối	Có lỗi?	Cách báo
Register File	Không	Mọi truy cập AXI4-Lite trả OKAY, kể cả offset dự trữ. RF chỉ *lưu* `ERR_CODE` do FSM cấp
Main FSM	Có	Nguồn duy nhất của `ERR_CODE`. Phát mã 1 từ kết quả kiểm tra descriptor; chuyển `o_rd_error` thành mã 2 hoặc 4 tùy pha, `o_wr_error` thành mã 3
DMA	Có	`o_rd_error` và `o_wr_error`, mỗi cái một xung. Không mang mã lỗi: FSM quyết mã theo pha đang chạy
Local SRAM	Không	Không có output lỗi. Vi phạm địa chỉ và vi phạm bank chỉ bị bắt bằng assertion mô phỏng
Systolic Array	Không	Không có output lỗi. Giả định M, N, K đã hợp lệ vì FSM kiểm trước khi phát `o_pe_start`
VPU	Không	Không có output lỗi. Tràn không xảy ra trên miền hợp lệ; bước tạo Y saturate thay vì báo lỗi

Lan truyền lỗi

- DMA của kênh lỗi ngừng phát burst mới, nhận hết beat R hoặc phản hồi B của burst đã cam kết, rồi phát một xung lỗi. Beat dữ liệu của burst lỗi vẫn được ghi vào scratchpad; nội dung đó không được dùng
- FSM ghi nhận lỗi vào một ô duy nhất gồm `err_v`, `err_idx`, `err_code`, `err_addr`; khi có lỗi mới, chỉ ghi đè nếu `err_idx` nhỏ hơn giá trị đang giữ. Nhờ vậy `ERR_CODE` và `ERR_DESC_ADDR` luôn ứng với descriptor lỗi sớm nhất theo thứ tự chuỗi
- Gọi f là descriptor lỗi sớm nhất. Sau khi có lỗi, FSM không bắt đầu pha nào của descriptor $\geq$ f (không lệnh DMA, không `o_pe_start` cho chúng); các descriptor đứng trước f vẫn chạy tới hết, kể cả STORE_C. Pha đang chạy chạy tới hết. Khi hệ yên — engine Đọc đã dừng và không chờ DMA, engine Tính không ở

C_RUN, engine Ghi rảnh và không còn job nào có chỉ số < f chờ tính hoặc ghi — FSM phát `o_err_pulse`. Điều kiện chính xác nằm ở mục Kết thúc tác vụ của tài liệu Main FSM; nhìn từ bên ngoài, lúc đó `i_rd_busy` = `i_wr_busy` = `i_pe_busy` = 0

- Array và VPU không bị thông báo về lỗi; chúng chỉ ngừng vì không nhận thêm `o_pe_start`. Nếu lỗi xảy ra trong lúc COMPUTE đang chạy, COMPUTE vẫn chạy tới `o_pe_done` rồi hệ thống mới yên
- Lỗi không bao giờ làm treo: mọi pha đang chạy đều có điều kiện kết thúc hữu hạn (burst AXI kết thúc bằng `rlast` hoặc `bvalid`; COMPUTE kết thúc sau số chu kỳ xác định)

6.5 Kiểm chứng ở mức hệ thống

Bảng 6.4: Điểm kiểm chứng mức hệ thống

ID	Nội dung
SYS-01	Nội dung DRAM sau một tác vụ khớp từng bit với mô hình tham chiếu ở mục 3.5, áp dụng lần lượt cho từng descriptor theo thứ tự chuỗi
SYS-02	Port top-level đúng 44 tín hiệu, đúng tên, độ rộng và chiều như bảng 1.6 và bảng 1.7; không có port nào khác
SYS-03	Mỗi descriptor được FETCH đúng một lần, bằng một burst 16 beat, theo đúng thứ tự chuỗi
SYS-04	Pha STORE_C ra DRAM đúng theo thứ tự descriptor; `DESC_DONE_CNT` tăng đơn điệu từng đơn vị
SYS-05	Tại mọi chu kỳ, mỗi bank của mỗi buffer có tối đa một bên truy cập (assertion AS-02 của SRAM)
SYS-06	Sau khi `o_pe_done`, đường ống VPU không còn bit valid nào (assertion AS-07 của VPU)
SYS-07	`o_done_pulse` và `o_err_pulse` không bao giờ cùng một chu kỳ; sau khi START được chấp nhận, đúng một trong ba bit BUSY, DONE, ERROR bằng 1 ở mọi lần đọc `STATUS`
SYS-08	Khi lỗi, `ERR_DESC_ADDR` và `DESC_DONE_CNT` ứng với descriptor lỗi sớm nhất; không pha nào của descriptor $\geq$ f đã bắt đầu
SYS-09	Chuỗi soft reset luôn kết thúc: từ `o_soft_req` tới `o_soft_clr` là hữu hạn chu kỳ, và sau đó `STATUS` = 0 mà không có DONE và không có ERROR
SYS-10	Kết quả không phụ thuộc mức độ chạy chồng: chạy cùng chuỗi với bus chậm (nhiều bubble AXI) và bus nhanh cho cùng nội dung DRAM
SYS-11	`DESC_PTR` không căn 64 byte kết thúc tác vụ với `ERR_CODE` = 1 mà **không** có giao dịch AXI master nào và không có truy cập scratchpad nào
SYS-12	Mọi truy cập AXI4-Lite, kể cả vào offset dự trữ 0x08 … 0x24, trả `s_axi_bresp` / `s_axi_rresp` = 2'b00

PHẠM VI

Bảng 6.4 là **yêu cầu kiểm chứng** của tài liệu thiết kế, không phải báo cáo kết quả. Bộ tài liệu baseline này là tài liệu; nó chưa được đối chiếu với RTL và chưa có mô phỏng nào được chạy.